

which is triply nested in the algorithm. This is essentially a matrix-matrix multiply, which takes $2d^3$ flops. If d is small enough that the three blocks A_{ij} , A_{ki} , and A_{kj} can all fit into cache at once, we can perform the entire operation without having to swap data into and out of cache. Since we are performing $2d^3$ flops on $3d^2$ data items, we have a ratio of $\frac{2}{3}d$ flops per data item. The larger d is, subject to the size constraint, the better our data use is. Similar benefits are obtained for the operations $A_{kk} \leftarrow \text{cholesky}(A_{kk})$ and $A_{kj} \leftarrow A_{kk}^{-T} A_{kj}$, since each requires $O(d^3)$ flops on $O(d^2)$ data items.

Exercise 1.4.49 Show that the implementation of $A_{kj} \leftarrow A_{kk}^{-T} A_{kj}$ via a sequence of forward substitutions requires $d^3 + O(d^2)$ flops if all three blocks are $d \times d$. $\square$

Let us also consider briefly the computation of the Cholesky factor on a parallel computer using (1.4.46). This may be worthwhile if the matrix is really huge. We continue to make the simplifying assumption that all of the blocks are $d \times d$. This implies that $n = ds$. If we increase d , we must decrease s , and vice versa. Let us focus on the part of the algorithm where most of the work is done. For each k , the operations

$$A_{ij} \leftarrow A_{ij} - A_{ki}^T A_{kj}$$

can be done simultaneously for $i = k + 1, \dots, s$, for $j = i, \dots, s$. Thus as many as about $s^2/2$ processors can be kept busy at once. This suggests that s should be made large (and, consequently, d small). On the other hand, a processor cannot perform the operation until it has received the data. Since $2d^3$ flops are performed on $3d^2$ data items, data movement is minimized by making d large. This suggests that d should be made large (and, consequently, s small). Thus we see that there are tradeoffs that keep us from making either d or s too large. Which choice of d and s is best will depend on the details of the parallel computer that is being used.

Exercise 1.4.50 Assuming $d \times d$ blocks in (1.4.46), calculate the fraction of the total work (reckoned in flops) that is spent (a) calculating Cholesky decompositions of the main-diagonal blocks, and (b) executing the instruction $A_{kj} \leftarrow A_{kk}^{-T} A_{kj}$. In each case your answer should be a function of s . What are the respective fractions when $s = 10$? when $s = 20$? $\square$

Proof of the Cholesky Decomposition Theorem

Having discussed the use of the Cholesky decomposition, having determined how to calculate the Cholesky factor, and having noted the numerous ways the computation can be organized, we now take time to prove the Cholesky Decomposition Theorem. We begin by recalling that a symmetric matrix A is positive definite if $x^T A x > 0$ for all nonzero $x \in \mathbb{R}^n$. We can use this property to prove a few simple propositions.

Proposition 1.4.51 *If A is positive definite, then $a_{ii} > 0$ for $i = 1, \dots, n$.*

Exercise 1.4.52 Prove Proposition 1.4.51. Do not use the Cholesky decomposition in your proof; we want to use this result to prove that the Cholesky decomposition exists. (Hint: Find a nonzero vector x such that $x^T A x = a_{ii}$.) $\square$

Proposition 1.4.53 Let A be positive definite, and consider a partition

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

in which A_{11} and A_{22} are square. Then A_{11} and A_{22} are positive definite.

Exercise 1.4.54 Prove Proposition 1.4.53. As in the previous exercise, do not use the Cholesky decomposition in your proof; use the fact that $x^T A x > 0$ for all nonzero x . $\square$

Propositions 1.4.51 and 1.4.53 are clearly closely related; 1.4.53 is (essentially) a generalization of 1.4.51. Can you think of a generalization that encompasses both of these results? What is the most general result you can think of? After you give this some thought, take a look at Exercise 1.4.61.

Proposition 1.4.55 If A and X are $n \times n$, A is positive definite, and X is nonsingular then the matrix $B = X^T A X$ is also positive definite.

Considering the special case $A = I$ (which is clearly positive definite), we see that this proposition is a generalization of Theorem 1.4.4.

Exercise 1.4.56 Prove Proposition 1.4.55. $\square$

The Cholesky Decomposition Theorem states that if A is positive definite, then there is a unique R such that R is upper triangular, the main diagonal entries of R are positive, and $A = R^T R$. We have already demonstrated uniqueness, so we only have to prove existence. We do so by induction on n . When $n = 1$, we have $A = [a_{11}]$. By Proposition 1.4.51, $a_{11} > 0$. Let $r_{11} = +\sqrt{a_{11}}$ and $R = [r_{11}]$. Then R has the desired properties. Moving on to the interesting part of the proof, we will show that every $n \times n$ positive definite matrix has a Cholesky factor, given that every $(n-1) \times (n-1)$ positive definite matrix does. Given an $n \times n$ positive definite A , partition A as we did in the development of the outer-product formulation of Cholesky's method:

$$A = \begin{bmatrix} a_{11} & b^T \\ b & \hat{A} \end{bmatrix}.$$

Proposition 1.4.51 guarantees that $a_{11} > 0$. Using (1.4.27) and (1.4.28) as a guide, define

$$\begin{aligned} r_{11} &= +\sqrt{a_{11}} \\ s &= r_{11}^{-1} b \\ \tilde{A} &= \hat{A} - s s^T. \end{aligned}$$

Then, as one easily checks,

$$A = \begin{bmatrix} r_{11} & 0 \\ s & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \tilde{A} \end{bmatrix} \begin{bmatrix} r_{11} & s^T \\ 0 & I \end{bmatrix}, \quad (1.4.57)$$

where I denotes the $(n-1) \times (n-1)$ identity matrix. The matrix

$$\begin{bmatrix} r_{11} & s^T \\ 0 & I \end{bmatrix}$$

is upper triangular and its main diagonal entries are nonzero, so it is nonsingular. Let us call its inverse X . Then if we let

$$B = \begin{bmatrix} 1 & 0 \\ 0 & \tilde{A} \end{bmatrix},$$

we have $B = X^T A X$, so B is positive definite, by Proposition 1.4.55. If we now apply Proposition 1.4.53 to B , we find that $\tilde{A}$ is positive definite. Since $\tilde{A}$ is $(n-1) \times (n-1)$, there is an upper triangular matrix $\tilde{R}$ whose main-diagonal entries are positive, such that $\tilde{A} = \tilde{R}^T \tilde{R}$, by the induction hypothesis. Therefore, using (1.4.57),

$$\begin{aligned} A &= \begin{bmatrix} r_{11} & 0 \\ s & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \tilde{R}^T \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \tilde{R} \end{bmatrix} \begin{bmatrix} r_{11} & s^T \\ 0 & I \end{bmatrix} \\ &= \begin{bmatrix} r_{11} & 0 \\ s & \tilde{R}^T \end{bmatrix} \begin{bmatrix} r_{11} & s^T \\ 0 & \tilde{R} \end{bmatrix} = R^T R, \end{aligned}$$

where R is upper triangular and has positive entries on the main diagonal. This completes the proof.

The matrix $\tilde{A} = \hat{A} - ss^T = \hat{A} - a_{11}^{-1}bb^T$, is called the *Schur complement* of a_{11} in A . The main business of the proof of the Cholesky Decomposition Theorem is to show that positive definiteness is inherited by the Schur complement. The argument is generalized in the following exercise.

Exercise 1.4.58 Let $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ be positive definite, and suppose A_{11} is $j \times j$ and A_{22} is $k \times k$. By Proposition 1.4.53, A_{11} is positive definite. Let R_{11} be the Cholesky factor of A_{11} , let $R_{12} = R_{11}^{-T} A_{12}$, and let $\tilde{A}_{22} = A_{22} - R_{12}^T R_{12}$. The matrix $\tilde{A}_{22}$ is called the *Schur complement* of A_{11} in A .

- Show that $\tilde{A}_{22} = A_{22} - A_{21} A_{11}^{-1} A_{12}$.
- Establish a decomposition of A that is similar to (1.4.57) and involves $\tilde{A}_{22}$.
- Prove that $\tilde{A}_{22}$ is positive definite.

□

Exercise 1.4.59 Write down a second proof of the existence of the Cholesky factor based on the decomposition established in the previous exercise. □

Exercise 1.4.60 Carefully prove by induction that the Cholesky decomposition is unique: Suppose $A = R^T R = S^T S$, where R and S are both upper-triangular matrices with

positive main-diagonal entries. Partition A , R , and S conformably and prove that the parts of S must equal the corresponding parts of R . $\square$

Exercise 1.4.61 This Exercise generalizes Propositions 1.4.51 and 1.4.53. Let A be an $n \times n$ positive definite matrix, let $j_1, j_2, \dots, j_k$ be integers such that $1 \leq j_1 < j_2 < \dots < j_k \leq n$, and let $\hat{A}$ be the $k \times k$ matrix obtained by intersecting rows $j_1, \dots, j_k$ with columns $j_1, \dots, j_k$. Prove that $\hat{A}$ is positive definite. $\square$

Exercise 1.4.62 Prove that if A is positive definite, then $\det(A) > 0$. $\square$

Complex Positive Definite Matrices

The next three exercises show how the results of this section can be extended to complex matrices. The set of complex n -vectors will be denoted $\mathbb{C}^n$. The *conjugate transpose* A^* of a complex $m \times n$ matrix A is the $n \times m$ matrix whose (i, j) entry is $\bar{a}_{ji}$. The bar denotes the complex conjugate. A is *hermitian* if $A = A^*$, that is, $a_{ij} = \bar{a}_{ji}$ for all i and j .

Exercise 1.4.63 (a) Prove that if A and B are complex $m \times n$ and $n \times p$ matrices, then $(AB)^* = B^*A^*$. (b) Prove that if A is hermitian, then x^*Ax is real for every $x \in \mathbb{C}^n$. (Hint: Let $\alpha = x^*Ax$. Then α is real if and only if $\alpha = \bar{\alpha}$. Think of α as a 1×1 matrix, and consider of the matrix α^* .) $\square$

If A is hermitian and satisfies $x^*Ax > 0$ for all nonzero $x \in \mathbb{C}^n$, then A is said to be *positive definite*.

Exercise 1.4.64 Prove that if M is any $n \times n$ nonsingular matrix, then M^*M is positive definite. $\square$

Exercise 1.4.65 Prove that if A is positive definite, then there exists a unique matrix R such that R is upper triangular and has positive (real!) main diagonal entries, and $A = R^*R$. This is the *Cholesky decomposition* of A . $\square$

All of the algorithms that we have developed in this section can easily be adapted to the complex case.

Additional Exercises

Exercise 1.4.66

- (a) The power drawn by an appliance (in watts) is equal to the product of the electromotive force (in volts) and the current (in amperes). Briefly, watts = volts $\times$ amps. Suppose a current I amperes passes through a resistor with resistance R ohms, causing a voltage drop of E volts. Show that the power dissipated by the resistor is (i) RI^2 watts, (ii) E^2/R watts.

- (b) Figure 1.9 illustrates loop currents passing through resistors with resistances R and S . Obviously the S ohm resistor draws power Sx_i^2 , which is positive unless

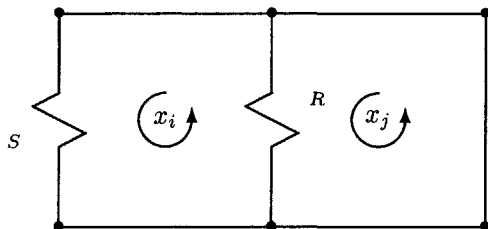


Fig. 1.9 Two loop currents

$x_i = 0$. Show that the power drawn by the R ohm resistor is $R(x_i - x_j)^2$. Deduce that this resistor draws positive power unless $x_i = x_j$, in which case it draws zero power. Show that the power drawn by this resistor can also be expressed as

$$\begin{bmatrix} x_i & x_j \end{bmatrix} \begin{bmatrix} R & -R \\ -R & R \end{bmatrix} \begin{bmatrix} x_i \\ x_j \end{bmatrix}.$$

- (c) Let

$$A = \begin{bmatrix} 9 & -5 \\ -5 & 8 \end{bmatrix},$$

the matrix of Example 1.2.8. Show that if $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ is the vector of loop currents, then the total power drawn by (all of the resistors in) the circuit is $x^T A x$. Show that A is positive definite.

- (d) Show that the 9×9 coefficient matrix of Exercise 1.2.19 is positive definite. (*Hint:* Show that the power drawn by the circuit is a sum of terms of the form Rx_k^2 and $R(x_i - x_j)^2$, and that this sum is positive unless all loop currents are zero. Then show that the power drawn by the circuit can also be expressed as $x^T A x$, where A is the coefficient matrix of the system.)

□

Exercise 1.4.67

- (a) The *conductance* C of a resistor (in siemens) is equal to the reciprocal of the resistance: $C = 1/R$. Show that if two nodes with voltages x_i and x_j are connected by a resistor with conductance C , the power drawn by the resistor

is $C(x_i - x_j)^2$, which can also be expressed as

$$\begin{bmatrix} x_i & x_j \end{bmatrix} \begin{bmatrix} C & -C \\ -C & C \end{bmatrix} \begin{bmatrix} x_i \\ x_j \end{bmatrix}.$$

This quantity is positive, unless $x_i = x_j$.

- (b) Consider the circuit in Figure 1.1, Example 1.2.6. Show that the power drawn by this circuit is a sum of terms of the form $C(x_i - x_j)^2$ and is positive unless all of the nodal voltages $x_1, \dots, x_6$ are equal.
- (c) Show that the power can be expressed as $x^T H x$, where H is a 6×6 symmetric matrix. This matrix is *positive semi-definite*, as $x^T H x \geq 0$ for all x .
- (d) Let A denote the coefficient matrix of the system in Example 1.2.6. Show that A is a submatrix of the matrix H from part (c). Deduce that A is positive definite. (*Hint*: Set $x_5 = x_6 = 0$ in the expression $x^T H x$.)
- (e) Show that the coefficient matrix of the circuit in Exercise 1.2.17 is positive definite.

□

Exercise 1.4.68

- (a) When a spring is stretched (or compressed) from its equilibrium position (0 meters) to some new position (x meters), the energy stored in the spring (strain energy) is equal to the work required to move the spring to its new position. This is $\int_0^x f(s) ds$ joules, the integral of force with respect to distance, where $f(s)$ is the force (in newtons) exerted against the spring that has been stretched to s meters. Show that for a linear spring with stiffness k newtons/meter, the strain energy is $\frac{1}{2} k x^2$ joules.
- (b) Show that if a spring is stretched or compressed by displacing its right and left endpoints by x_2 and x_1 meters, respectively, the strain energy of the spring is $\frac{1}{2} k (x_1 - x_2)^2$ joules, which is positive unless $x_1 = x_2$. Show that the strain energy can also be written in the form

$$\frac{1}{2} \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} k & -k \\ -k & k \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

- (c) Show that the total strain energy of the mass-spring system in Example 1.2.10 is a sum of terms of the form $k x_i^2$ and $k(x_i - x_{i+1})^2$ and is positive unless all of the displacements are zero.
- (d) Let A be the coefficient matrix of the mass-spring system of Example 1.2.10. Show that the strain energy of the system is $\frac{1}{2} x^T A x$. Deduce that A is positive definite.

- (e) Show that the coefficient matrix of the mass-spring system of Exercise 1.2.20 is positive definite.

□

Exercise 1.4.69 Let v be a vector whose entries represent some statistical data. For example, v could be a vector with 365 entries representing the high temperature in Seattle on 365 consecutive days. We can normalize this vector by computing its mean and subtracting the mean from each of the entries to obtain a new vector with mean zero. Suppose now we have such a normalized vector. Then the *variance* of v is $\sum_{i=1}^n v_i^2$. This nonnegative number gives a measure of the variation in the data. Notice that if we think of v as a column vector, the variance can be expressed as $v^T v$. Now let v and w be two vectors with mean zero and variance (normalized to be) one. Then the *correlation* of v and w is defined to be $\sum_{i=1}^n v_i w_i = w^T v = v^T w$. This number, which can be positive or negative, measures whether the data in v and w vary with or against one another. For example, the temperatures in Seattle and Tacoma should have a positive correlation, while the temperatures in Seattle and Hobart, Tasmania, should have a negative correlation. Now consider k vectors $v_1, \dots, v_k$ with mean zero and variance one. The *correlation matrix* C of the data $v_1, \dots, v_k$ is the $k \times k$ matrix whose (i, j) entry is $c_{ij} = v_j^T v_i$, the correlation of v_i with v_j . Show that C is a symmetric matrix whose main-diagonal entries are all ones. Show that $C = V^T V$ for some appropriately constructed (nonsquare) matrix V . Show that C is positive definite if the vectors $v_1, \dots, v_k$ are linearly independent. Show that if $v_1, \dots, v_k$ are linearly dependent, then C is not positive definite, but it is positive semidefinite, i.e. $x^T C x \geq 0$ for all x . □

Exercise 1.4.70 Draw pictures that demonstrate that

$$\int_0^n i^2 \, di \leq \sum_{i=0}^n i^2 \leq \int_0^{n+1} i^2 \, di.$$

Deduce that

$$\sum_{i=0}^n i^2 = \frac{n^3}{3} + O(n^2).$$

□

Exercise 1.4.71 Prove by induction on n that

$$\sum_{i=0}^n i^2 = \frac{n(n + \frac{1}{2})(n + 1)}{3}.$$

□

Exercise 1.4.72 Figure out what the following MATLAB code does.

```
n = 100;
for jay = 1:4
    if jay > 1; oldtime = time ; end
    M = randn(n);
    A = M'*M;
    t = cputime;
    R = chol(A);
    matrixsize = n
    time = cputime - t
    if jay > 1; ratio = time/oldtime, end
    n = 2*n;
end
```

If you put these instructions into a file named, say, zap.m, you can run the program by typing zap from the MATLAB command line. The functions randn, cputime, and chol are built-in MATLAB functions, so you can learn about them by typing help randn, etc. You might find it useful to type more on before typing help {topic}.

- Does the code produce reasonable values of ratio when you run it? What value would you expect in theory? Depending on the speed of the machine on which you are running MATLAB, you may want to adjust n or the number of times the jay loop is executed.
- Why is the execution time of this code (i.e. the time you sit waiting for the answers) so much longer than the times that are printed out?

□

Exercise 1.4.73 Write Fortran subroutines to implement Cholesky's method in (a) inner-product form, (b) outer-product form, (c) bordered form. If you have already written a forward-substitution routine, you can use it in part (c). Your subroutines should operate only on the main diagonal and upper-triangular part of A , and they should overwrite A with R . They should either return R or set a warning flag indicating that A is not positive definite. Try out your routines on the following examples.

$$\begin{aligned}
 \text{(i) } A &= \begin{bmatrix} 36 & 30 & 24 \\ 30 & 34 & 26 \\ 24 & 26 & 21 \end{bmatrix} \\
 \text{(ii) } A &= \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \\ -1 & 2 & -2 & 2 & -2 \\ 1 & -2 & 3 & -3 & 3 \\ -1 & 2 & -3 & 4 & -4 \\ 1 & -2 & 3 & -4 & 5 \end{bmatrix} \\
 \text{(iii) } A &= \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 10 \\ 3 & 10 & 16 \end{bmatrix} \quad \text{cf. Example 1.4.20.}
 \end{aligned}$$

You might like to devise some additional examples. The easy way to do this is to write down R first and then multiply R^T by R to get A . With the help of MATLAB you can generate larger matrices. Use the MATLAB `save` command to export a matrix to an ASCII file. Type `help save` for details. $\square$

Exercise 1.4.74 Write a Fortran program that solves positive definite systems $Ax = b$ by calling subroutines to (a) calculate the Cholesky factor, (b) perform forward substitution, and (c) perform back substitution. Try out your program on the following problems.

$$(i) \begin{bmatrix} 36 & -30 & 24 \\ -30 & 34 & -26 \\ 24 & -26 & 21 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 12 \\ -7 \end{bmatrix}$$

$$(ii) \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 2 & 2 \\ 1 & 2 & 3 & 3 & 3 \\ 1 & 2 & 3 & 4 & 4 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 5 \\ 9 \\ 12 \\ 14 \\ 15 \end{bmatrix}$$

You might like to make some additional examples. You can use MATLAB to help you build larger examples, as suggested in the previous exercise. $\square$

1.5 BANDED POSITIVE DEFINITE SYSTEMS

Large systems of equations occur frequently in applications, and large systems are usually sparse. In this section we will study a simple yet very effective scheme for applying Cholesky's method to large, positive definite systems of equations that are banded or have an envelope structure. This method is in widespread use and, as we shall see, it can yield enormous savings in computer time and storage space. However, it is not necessarily the most efficient scheme. More sophisticated sparse matrix methods are discussed briefly in Section 1.6. For details see [30] and [21], for example. For extremely large systems, iterative methods are preferred. We discuss iterative methods for sparse linear systems in Chapter 7.

A matrix A is *banded* if there is a narrow band around the main diagonal such that all of the entries of A outside of the band are zero, as shown in Figure 1.10. More precisely, if A is $n \times n$, and there is an $s \ll n$ such that $a_{ij} = 0$ whenever $|i - j| > s$, then all of the nonzero entries of A are confined to a band of $2s + 1$ diagonals centered on the main diagonal. We say that A is *banded* with *band width* $2s + 1$. Since we are concerned with symmetric matrices in this section, we only need half of the band. Since $a_{ij} = 0$ whenever $i - j > s$, there is a band of s diagonals above the main diagonal that, together with the main diagonal, contains all of the nonzero entries of A . We say that A has *semiband width* s .

Example 1.5.1 Consider the mass-spring system depicted in Figure 1.11. This is exactly the system that we discussed in Exercise 1.2.20. There are n carts attached

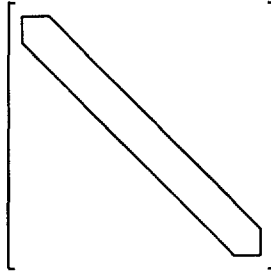


Fig. 1.10 Banded matrix: All entries outside of the band are zero.

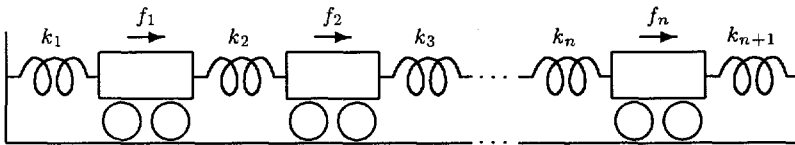


Fig. 1.11 System of n masses

by springs. If forces are applied to the carts, we can calculate their displacements x_i by solving a system $Ax = b$ of n equations in n unknowns. Since the i th cart is directly attached only to the two adjacent carts, the i th equation involves only the unknowns x_{i-1} , x_i , and x_{i+1} . Thus its form is

$$a_{i,i-1}x_{i-1} + a_{i,i}x_i + a_{i,i+1}x_{i+1} = b_i,$$

and $a_{ij} = 0$ whenever $|i - j| > 1$. This is an extreme example of a banded coefficient matrix. The band width is 3 and the semiband width is 1. Such matrices are called *tridiagonal*. $\square$

Example 1.5.2 Consider a 100×100 system of equations $Ax = b$ associated with the grid depicted in Figure 1.12. The i th grid point has one equation and one unknown associated with it. For example, the unknown could be a nodal voltage (or a displacement, a pressure, a temperature, a hydraulic head, ...). Assume that the i th equation involves only the unknowns associated with the i th grid point and the grid points that are directly connected to it. For example, the 34th equation involves only the unknowns x_{24} , x_{33} , x_{34} , x_{35} , and x_{44} . This means that in the 34th row of A , only the entries $a_{34,24}$, $a_{34,33}$, $a_{34,34}$, $a_{34,35}$, and $a_{34,44}$ are nonzero. The other 95 are zero. Clearly the same is true for every other equation; no row of A contains more than five nonzero entries. Thus the matrix is very sparse. It is also banded, if the equations and unknowns are numbered as shown in the figure. Clearly $a_{ij} = 0$ if $|i - j| > 10$. Thus the system is 100×100 with a semiband width of 10. $\square$

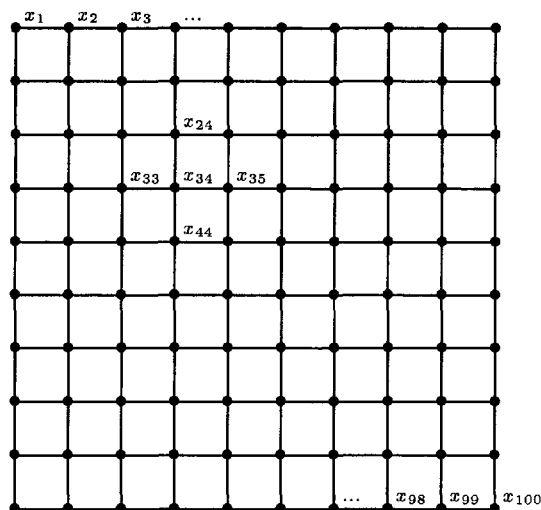


Fig. 1.12 “Large” grid

Exercise 1.5.3 Make a rough sketch of the matrix A of Example 1.5.2, noting where the zeros and nonzeros lie. Notice that even within the band, most of the entries are zero. Most large application problems have this feature. $\square$

Exercise 1.5.4 Modern applications usually involve matrices that are much larger than the one discussed in Example 1.5.2. Figure 1.12 depicts a 10×10 network of nodes. Imagine an $m \times m$ network with $m \gg 10$. How many equations does the resulting system have? How many nonzeros does each row of the matrix have? What is the bandwidth of the system, assuming that the equations and unknowns are numbered as depicted in Figure 1.12? Answer these questions in general and also in the specific cases (a) $m = 100$, (b) $m = 1000$. $\square$

Notice that the bandedness depends on how the nodes are numbered. If, for example, nodes 2 and 100 are interchanged in Figure 1.12, the resulting matrix is not banded, since $a_{100,1}$ and $a_{1,100}$ are nonzero. However it is still sparse; the number of nonzero entries in the matrix does not depend on how the nodes are ordered.

If a network is regular, it is easy to see how to number the nodes to obtain a narrow band. Irregular networks can also lead to banded systems, but it will usually be more difficult to decide how the nodes should be numbered.

Banded positive definite systems can be solved economically because it is possible to ignore the entries that lie outside of the band. For this it is crucial that the Cholesky factor inherits the band structure of the original matrix. Thus we can save storage

space by using a data structure that stores only the semiband of A . R can be stored over A . Just as importantly, computer time is saved because all operations involving entries outside of the band can be skipped. As we shall soon see, these savings are substantial.

Instead of analyzing banded systems, we will introduce a more general idea, that of the envelope of a matrix. This will increase the generality of the discussion while simplifying the analysis. The *envelope* of a symmetric or upper-triangular matrix A is a set of ordered pairs (i, j) , $i < j$, representing element locations in the upper triangle of A , defined as follows: (i, j) is in the envelope of A if and only if $a_{kj} \neq 0$ for some $k \leq i$. Thus if the first nonzero entry of the j th column is a_{mj} and $m < j$, then (m, j) , $(m + 1, j)$, $\dots$, $(j - 1, j)$ are the members of the envelope of A from the j th column.

The crucial theorem about envelopes (Theorem 1.5.7) states that if R is the Cholesky factor of A , then R has the same envelope as A . Thus A can be stored in a data structure that stores only its main diagonal and the entries in its envelope, and R can be stored over A . All operations involving the off-diagonal entries lying outside of the envelope can be skipped. If the envelope is small, substantial savings in computer time and storage space are realized. Banded matrices have small envelopes. A simple example of an unbanded matrix with a small envelope is

$$\begin{bmatrix} 2 & -1 & 0 & \cdots & 0 & 0 & -1 \\ -1 & 2 & -1 & \ddots & & 0 & 0 \\ 0 & -1 & 2 & \ddots & & & 0 \\ \vdots & \ddots & \ddots & \ddots & & & \vdots \\ 0 & & & & 2 & -1 & 0 \\ 0 & 0 & & & -1 & 2 & -1 \\ -1 & 0 & 0 & \cdots & 0 & -1 & 2 \end{bmatrix}, \quad (1.5.5)$$

which was obtained from discretization of an ordinary differential equation with a periodic boundary condition.

Exercise 1.5.6 Identify the envelope of the matrix in (1.5.5). Assuming the matrix is $n \times n$, approximately what fraction of the upper triangle of the matrix lies within the envelope? $\square$

Like the band width, the envelope of a matrix depends on the order in which the equations and unknowns are numbered. Often it is easy to see how to number the nodes to obtain a reasonably small envelope. For those cases in which it is hard to tell how the nodes should be ordered, there exist algorithms that attempt to minimize the envelope in some sense. For example, see the discussion of the reverse Cuthill-McKee algorithm in [30].

Theorem 1.5.7 Let A be positive definite, and let R be the Cholesky factor of A . Then R and A have the same envelope.

Proof. Consider the bordered form of Cholesky's method. At the j th step we solve a system

$$R_{j-1}^T h = c,$$

where $c \in \mathbb{R}^{j-1}$ is the portion of the j th column of A lying above the main diagonal, and h is the corresponding portion of R . (See equations (1.4.36) and (1.4.37).) Let $\hat{c} \in \mathbb{R}^{s_j}$ be the portion of c that lies in the envelope of A . Then

$$c = \begin{bmatrix} 0 \\ \hat{c} \end{bmatrix}.$$

In Section 1.3 we observed that if c has leading zeros, then so does h :

$$h = \begin{bmatrix} 0 \\ \hat{h} \end{bmatrix},$$

where $\hat{h} \in \mathbb{R}^{s_j}$. See (1.3.6) and the accompanying discussion. It follows immediately that the envelope of R is contained in the envelope of A . Furthermore it is not hard to show that the first entry of $\hat{h}$ is nonzero. Thus the envelope of R is exactly the envelope of A . $\square$

Corollary 1.5.8 *Let A be a banded, positive definite matrix with semiband width s . Then its Cholesky factor R also has semiband width s .*

Referring to the notation in the proof of Theorem 1.5.7, if $\hat{c} \in \mathbb{R}^{s_j}$, then the cost of the arithmetic in the j th step of Cholesky's method is essentially equal to that of solving an $s_j \times s_j$ lower-triangular system, that is, s_j^2 flops. (See the discussion following (1.3.6).) If the envelope is not exploited, the cost of the j th step is j^2 flops. To get an idea of the savings that can be realized by exploiting the envelope structure of a matrix, consider the banded case. If A has semiband width s , then the portion of the j th row that lies in the envelope has at most s entries, so the flop count for the j th step is about s^2 . Since there are n steps in the algorithm, the total flop count is about ns^2 .

Exercise 1.5.9 Let R be an $n \times n$ upper-triangular matrix with semiband width s . Show that the system $Rx = y$ can be solved by back substitution in about $2ns$ flops. An analogous result holds for lower-triangular systems. $\square$

Example 1.5.10 The matrix of Example 1.5.2 has $n = 100$ and $s = 10$. If we perform a Cholesky decomposition using a program that does not exploit the band structure of the matrix, the cost of the arithmetic is about $\frac{1}{3}n^3 \approx 3.3 \times 10^5$ flops. In contrast, if we do exploit the band structure, the cost is about $ns^2 = 10^4$ flops, which is about 3% of the previous figure. In the forward and back substitution steps, substantial but less spectacular savings are achieved. The combined arithmetic cost of forward and back substitution without exploiting the band structure is about $2n^2 = 2 \times 10^4$ flops. If the band structure is exploited, the flop count is about $4ns = 4 \times 10^3$, which is 20% of the previous figure.

If the matrix is stored naively, space for $n^2 = 10,000$ numbers is needed. If only the semiband is stored, space for not more than $n(s+1) = 1100$ numbers is required. $\square$

The results of Example 1.5.10, especially the savings in flops in the Cholesky decomposition, are already impressive, even though the matrix is not particularly large. Much more impressive results are obtained if larger matrices are considered, as the following exercise shows.

Exercise 1.5.11 As in Exercise 1.5.4, consider the banded system of equations arising from an $m \times m$ network of nodes like Figure 1.12 but larger, with the nodes numbered by rows, as in Figure 1.12.

- (a) For the case $m = 100$ (for which $n = 10^4$) calculate the cost of solving the system $Ax = b$ (Cholesky decomposition plus forward and back substitution) with and without exploiting the band structure. Show that exploiting the band structure cuts the flop count by a factor of several thousand. Show that if only the semiband is stored, the storage space required is only about 1% of what would be required to store the matrix naively.
- (b) Repeat part (a) with $m = 1000$.

$\square$

Savings such as these can make the difference between being able to solve a large problem and not being able to solve it.

The following exercises illustrate another important feature of banded and envelope matrices: The envelope structure of A is *not* inherited by A^{-1} . In fact, it is typical of sparse matrices that the inverse is not sparse. Thus it is highly uneconomical to solve a sparse system $Ax = b$ by finding the inverse and computing $x = A^{-1}b$.

Exercise 1.5.12 Consider a mass-spring system as in Figure 1.11 with six carts. Suppose each spring has a stiffness $k_i = 1$ newton/meter.

- (a) Set up the tridiagonal, positive definite, coefficient matrix A associated with this problem.
- (b) Use the MATLAB `chol` command to calculate the Cholesky factor R . Notice that R inherits the band structure of A . (To learn an easy way to enter this particular A , type `help toeplitz`.)
- (c) Use the MATLAB `inv` command to calculate A^{-1} . Notice that A^{-1} does not inherit the band structure of A .

$\square$

Exercise 1.5.13 In the previous exercise, the matrix A^{-1} is full; none of its entries are zero.

- (a) What is the physical significance of this fact? (Think of the equation $x = A^{-1}b$, especially in the case where only one entry, say the j th, of b is nonzero. If the

(i, j) entry of A^{-1} were zero, what would this imply? Does this make physical sense?)

- (b) The entries of A^{-1} decrease in magnitude as we move away from the main diagonal? What does this mean physically?

□

Exercise 1.5.14 Consider the linear system $Ax = b$ from Exercise 1.2.19. The matrix A is banded and positive definite.

- (a) Use MATLAB to compute the Cholesky factor R . Observe that the envelope is preserved.
- (b) Use MATLAB to calculate A^{-1} . Observe that A^{-1} is full. (What is the physical significance of this?)

□

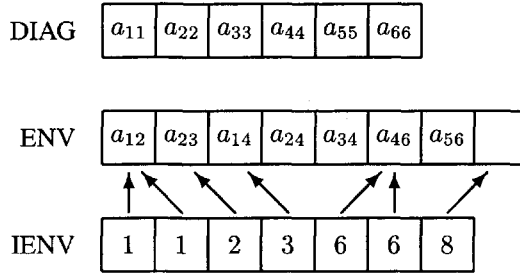
Envelope Storage Scheme

A fairly simple data structure can be used to store the envelope of a coefficient matrix. We will describe the scheme from [30]. A one-dimensional real array **DIAG** of length n is used to store the main diagonal of the matrix. A second one-dimensional real array **ENV** is used to store the envelope by columns, one after the other. A third array **IENV**, an integer array of length $n + 1$, is used to store pointers to **ENV**. Usually $\text{IENV}(J)$ names the position in **ENV** of the first (nonzero) entry of column J of the matrix. However, if column J contains no nonzero entries above the main diagonal, then $\text{IENV}(J)$ points to column $J + 1$ instead. Thus the absence of nonzero entries in column J above the main diagonal is signaled by $\text{IENV}(J) = \text{IENV}(J + 1)$. $\text{IENV}(n + 1)$ points to the first storage location after the envelope. These rules can be expressed more succinctly (and more accurately) as follows: $\text{IENV}(1) = 1$ and $\text{IENV}(J + 1) - \text{IENV}(J)$ equals the number of elements from column J of the matrix that lie in the envelope.

Example 1.5.15 The matrix

$$\begin{bmatrix} a_{11} & a_{12} & 0 & a_{14} & 0 & 0 \\ & a_{22} & a_{23} & a_{24} & 0 & 0 \\ & & a_{33} & a_{34} & 0 & 0 \\ & & & a_{44} & 0 & a_{46} \\ & & & & a_{55} & a_{56} \\ & & & & & a_{66} \end{bmatrix}$$

is stored as follows using the envelope scheme:



□

When the envelope storage scheme is used, certain formulations of the Cholesky decomposition, forward-substitution, and back-substitution algorithms are much more appropriate than others. For example, we would not want to use the outer-product formulation of Cholesky's method, because that algorithm operates on (the upper triangle of) A by rows. In the envelope storage scheme A is stored by columns; rows are hard to access. The inner-product formulation is inappropriate for the same reason.⁴ From the proof of Theorem 1.5.7 it is clear that the bordered form of Cholesky's method is appropriate. At each step virtually all of the work goes into solving a lower-triangular system $R_{i-1}^T h = c$, where

$$c = \begin{bmatrix} 0 \\ \hat{c} \end{bmatrix} \quad \text{and} \quad h = \begin{bmatrix} 0 \\ \hat{h} \end{bmatrix}.$$

If we partition R_{i-1}^T conformably,

$$R_{i-1}^T = \begin{bmatrix} H_{11} & 0 \\ H_{21} & H_{22} \end{bmatrix},$$

the equation $R_{i-1}^T h = c$ reduces to $H_{22} \hat{h} = \hat{c}$. H_{22} is a lower-triangular matrix consisting of rows and columns $i - s_i$ through $i - 1$ of R^T . A subroutine can be used to solve $H_{22} \hat{h} = \hat{c}$. What is needed is a forward-substitution routine that solves systems of the form $\hat{R}^T \hat{h} = \hat{c}$, where $\hat{R}$ is a submatrix of R consisting of rows and columns j through k , where j and k can be any integers satisfying $1 \leq j \leq k \leq n$. Since R^T , hence $\hat{R}^T$, is stored by rows, the appropriate formulation of forward substitution is the row-oriented version. This subroutine can also be used with $j = 1$ and $k = n$ to perform the forward-substitution step ($R^T y = b$) after the Cholesky decomposition has been completed. Finally, a back-substitution routine is needed to solve $Rx = y$. Since R is stored by columns, we use the column-oriented version.

Exercise 1.5.16 Write a set of three Fortran subroutines to solve positive definite systems, using the envelope storage scheme:

⁴The inner-product formulation accesses A by both rows and columns.

- (a) Row-oriented forward-substitution routine, capable of solving systems $\hat{R}^T \hat{h} = \hat{c}$, where $\hat{R}$ is a submatrix of R consisting of rows and columns j through k , $1 \leq j \leq k \leq n$.
- (b) Cholesky decomposition routine, bordered form, which calls the forward-substitution routine to do most of the work.
- (c) column-oriented back-substitution routine.

Write a main program that allocates storage, handles input and output, and calls the subroutines to solve positive definite systems $Ax = b$. Test your programs using the test problems given below.

For storage you will need the arrays DIAG, ENV, and IENV discussed above and one additional real array of length n that holds b initially, gets changed to y during the forward-substitution step, and finally gets changed to x during the back-substitution step. The arrays DIAG and ENV contain A initially and get changed to R as the Cholesky decomposition is computed. This is all the storage space that is needed (except, of course, for the space occupied by the program itself). Test problems:

$$(a) A = \begin{bmatrix} 4 & 0 & 6 & 0 & 2 \\ 0 & 1 & 3 & 0 & 2 \\ 6 & 3 & 19 & 2 & 6 \\ 0 & 0 & 2 & 5 & -5 \\ 2 & 2 & 6 & -5 & 16 \end{bmatrix} \quad (i) b = \begin{bmatrix} 12 \\ 6 \\ 36 \\ 2 \\ 21 \end{bmatrix} \quad (ii) b = \begin{bmatrix} 12 \\ 4 \\ 26 \\ -8 \\ 27 \end{bmatrix}$$

The Cholesky decomposition only has to be done once. *Solution:*

$$R = \begin{bmatrix} 2 & 0 & 3 & 0 & 1 \\ & 1 & 3 & 0 & 2 \\ & & 1 & 2 & -3 \\ & & & 1 & 1 \\ & & & & 1 \end{bmatrix}$$

$$(i) y = \begin{bmatrix} 6 \\ 6 \\ 0 \\ 2 \\ 1 \end{bmatrix} \quad x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad (ii) y = \begin{bmatrix} 6 \\ 4 \\ -4 \\ 0 \\ 1 \end{bmatrix} \quad x = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \\ 1 \end{bmatrix}$$

$$(b) A = \begin{bmatrix} 16 & 4 & 4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 5 & -3 & 6 & 0 & 0 & 0 & 10 & 0 & 0 \\ 4 & -3 & 6 & -8 & 0 & 2 & 0 & -9 & 0 & 0 \\ 0 & 6 & -8 & 17 & 8 & -2 & 0 & 13 & 0 & 0 \\ 0 & 0 & 0 & 8 & 17 & 3 & 2 & -1 & 0 & 0 \\ 0 & 0 & 2 & -2 & 3 & 15 & 1 & -3 & 3 & 0 \\ 0 & 0 & 0 & 0 & 2 & 1 & 21 & -4 & -7 & 0 \\ 0 & 10 & -9 & 13 & -1 & -3 & -4 & 32 & -1 & 4 \\ 0 & 0 & 0 & 0 & 0 & 3 & -7 & -1 & 10 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 & 0 & 24 \end{bmatrix} \quad b = \begin{bmatrix} 36 \\ 109 \\ -76 \\ 188 \\ 141 \\ 113 \\ 68 \\ 281 \\ 51 \\ 272 \end{bmatrix}$$

- (c) Use your program to show that

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 2 & 13 & 16 \\ 4 & 16 & 10 \end{bmatrix}$$

is not positive definite.

- (d) Think about how your subroutines could be used to calculate the inverse of a positive definite matrix. Calculate A^{-1} , where

$$A = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}.$$

It turns out that the entries of A^{-1} are all integers. Notice that your computed solution suffers from significant roundoff errors. This is because A is (mildly) ill conditioned. This is the 3×3 member of a famous family of ill-conditioned matrices called *Hilbert matrices*; the condition gets rapidly worse as the size of the matrix increases. We will discuss ill-conditioned matrices in Chapter 2.

□

1.6 SPARSE POSITIVE DEFINITE SYSTEMS

If one compares a sparse matrix A with its Cholesky factor R , one normally finds that R has many more nonzero entries than the upper half of A does. The “new” nonzero entries are called *fill* or *fill-in*. How much fill one gets depends on how the equations are ordered.

Example 1.6.1 An arrowhead matrix like

$$A = \begin{bmatrix} 7 & 0 & 0 & 1 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

suffers no fill-in during the Cholesky decomposition. Its Cholesky factor is

$$R = \begin{bmatrix} \sqrt{7} & 0 & 0 & 1/\sqrt{7} \\ & \sqrt{3} & 0 & 1/\sqrt{3} \\ & & \sqrt{2} & 1/\sqrt{2} \\ & & & 1/\sqrt{42} \end{bmatrix},$$

which has as many zeros above the main diagonal as A has. Now consider the matrix

$$\hat{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 0 & 0 \\ 1 & 0 & 3 & 0 \\ 1 & 0 & 0 & 7 \end{bmatrix},$$

obtained from A by reversing the order of the rows and columns. This reversed arrowhead matrix has the Cholesky factor

$$\hat{R} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ & 1 & -1 & -1 \\ & & 1 & -2 \\ & & & 1 \end{bmatrix},$$

which is completely filled in. Obviously the same will happen to any arrowhead matrix; the pattern of nonzeros is what matters, not their exact values. Furthermore, there is nothing special about the case $n = 4$; we can build arbitrarily large arrowhead matrices. Notice that A has a small envelope that contains no nonzero entries, whereas $\hat{A}$ has a large envelope that includes many zeros. Theorem 1.5.7 guarantees that the envelope of R will be no bigger than the envelope of A , but it says nothing about the fate of zeros within the envelope. Usually zeros within the envelope of A will turn into nonzeros in R . $\square$

The key to making the sparse Cholesky decomposition economical is to keep the fill under control. Ideally one would like to find the reordering that minimizes fill. As it turns out, this is a difficult problem (there are $n!$ orderings to consider). The solution seems to be beyond reach. Fortunately there are several practical algorithms that do a reasonable job of keeping the fill-in under control [30]. We mention two of the more popular methods. The *reverse Cuthill-McKee* algorithm attempts to make the bandwidth small. The *minimum-degree* algorithm tries to minimize fill-in by analyzing the sparsity pattern of the matrix using graph-theoretic methods. Bandwidth is ignored. For descriptions of these two methods see [30]. We will explore their performance by means of some MATLAB examples.

MATLAB has considerable support for sparse matrix computations. There is an easy-to-use sparse matrix data structure. Most of the operations that are available for full (i.e. non-sparse) matrices can be applied to sparse matrices as well. For example, if A is a positive-definite matrix stored in the sparse format, the command `R = chol(A)` gives the Cholesky factor of A , also stored in the sparse format. There are also numerous commands that apply strictly to sparse matrices. Type `help sparsfun` in MATLAB for a list of these.

Example 1.6.2 A nice example of a sparse matrix that's not too big is `bucky`, which is the incidence matrix of the Bucky Ball (soccer ball). The Bucky Ball is a polyhedron with 60 vertices, 32 faces (12 pentagons and 20 hexagons), and 30 edges. An incidence matrix is obtained by numbering the vertices and building a 60×60 matrix whose (i, j) entry is 1 if vertices i and j are connected by an edge and 0 otherwise. Since each vertex is connected to exactly three others, each row of the Bucky Ball matrix has exactly three nonzero entries. This matrix is not positive definite; its main diagonal entries are all zero. We can make a positive definite matrix by adding $3I$. We did this in MATLAB by typing `A = bucky + 3*speye(60)`. (The command `speye(60)` gives a 60×60 identity matrix, stored as a sparse matrix.) The resulting A is a 60×60 positive definite matrix with four nonzero

entries in each row and each column. For such a small matrix we do not really need to use sparse matrix techniques, but it is a good example with which to begin.

Different numberings of the vertices of the Bucky Ball correspond to different orderings of the rows/columns of A . Figure 1.13 shows the pattern of nonzeros in

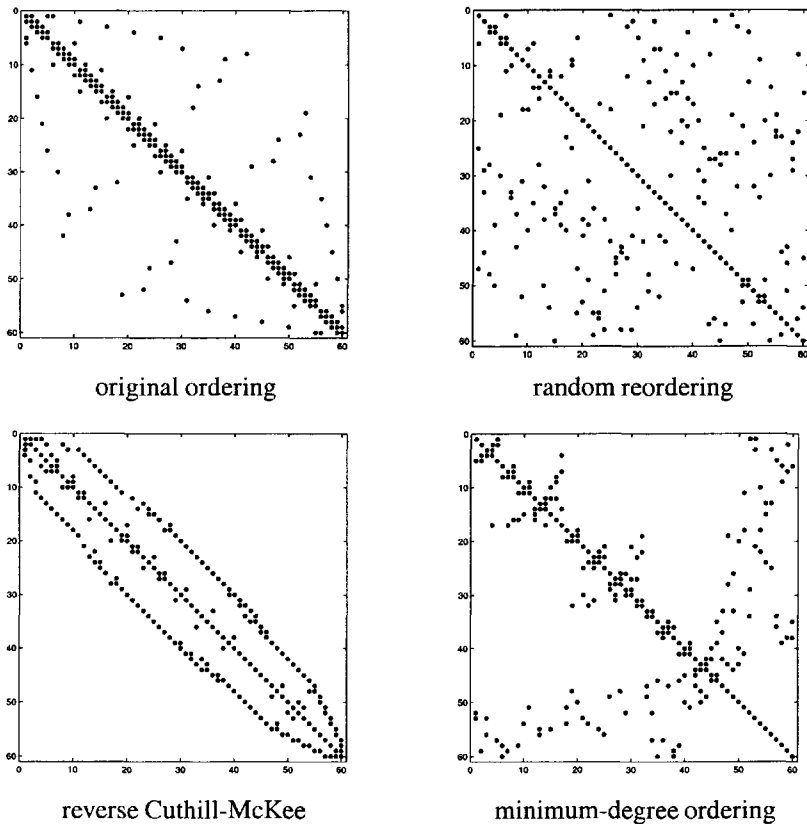


Fig. 1.13 Spy plots of several orderings of modified Bucky Ball matrix

A in the “original” ordering specified by MATLAB and in three reorderings. Plots of this type are called *spy plots* in MATLAB and are generated by the command `spy(A)`. Each of the four plots in Figure 1.13 has 240 dots, corresponding to the 240 nonzero entries of A . We note that the reverse Cuthill-McKee ordering gathers the nonzeros into a band, whereas the minimum-degree ordering does not. We calculated the Cholesky factor of A and each of the reorderings and displayed their spy plots in Figure 1.14. The number of nonzero entries, indicating the amount of fill, is also listed for each case. In the case of the reverse Cuthill-McKee ordering, the fill is restricted to a narrow band, but notice that the band is now almost completely filled in. In fact, the total amount of fill-in incurred by the reverse Cuthill-McKee ordering is not significantly less than it was for the original ordering or for the random

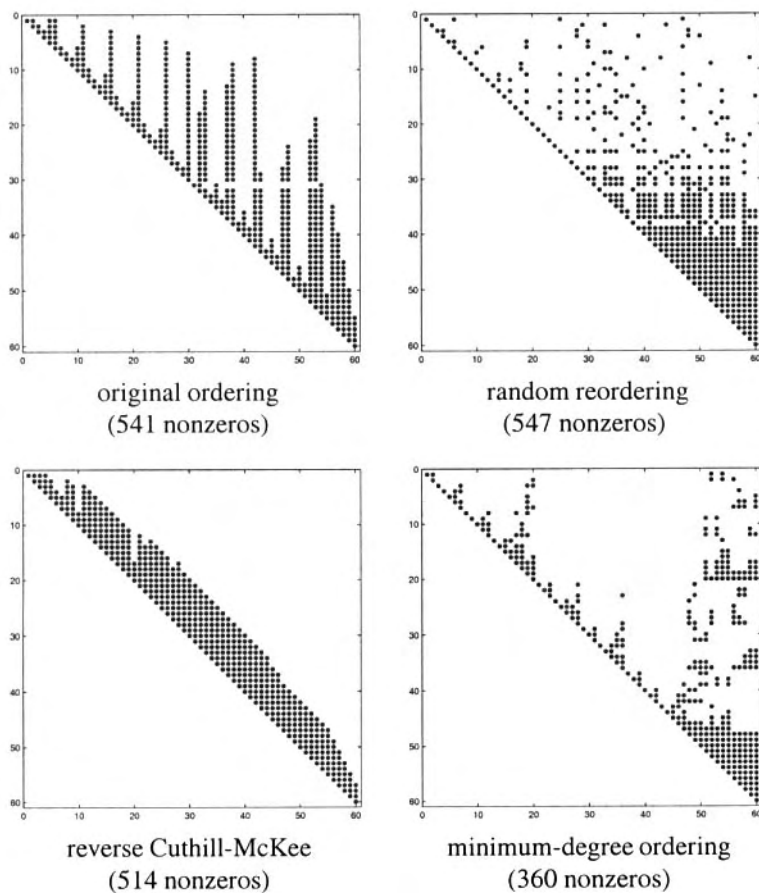


Fig. 1.14 Spy plots of Cholesky factors of reorderings of modified Bucky Ball matrix

reordering. In subsequent exercises you will see that for larger matrices significant reduction of fill is eventually achieved by reverse Cuthill-McKee.

The most striking revelation of Figure 1.14 is that the minimum-degree algorithm cuts fill significantly. The Cholesky factor from the minimum degree ordering has 154 fewer nonzero entries than the reverse Cuthill-McKee Cholesky factor has. Of course, one example does not prove anything, but tests on larger problems confirm that the minimum degree algorithm does do better than reverse Cuthill-McKee on a wide variety of problems. However, effective exploitation of the good fill properties of the minimum-degree ordering requires use of a more flexible data structure for sparse matrices, since the fill is not restricted to a narrow band. In contrast, if we use the reverse Cuthill-McKee ordering, we can use a simple banded or envelope scheme that accommodates the fill automatically. $\square$

MATLAB's sparse matrix data structure is quite simple. Three numbers, m , n , and nz specify the number of rows, columns, and nonzero entries of the array. The information about the matrix entries and their locations is stored in three lists (one-dimensional arrays), i , j , and s , of length (at least) nz . Arrays i and j have integer entries, and s has (double precision) real entries. For each $k \leq nz$, the sparse matrix A has a nonzero entry equal to $s(k)$ in position $(i(k), j(k))$.⁵ When the MATLAB command $R = \text{chol}(A)$ is used to compute the Cholesky factor of the sparse matrix A , MATLAB has to allocate space for the sparse matrix R . In doing so it must take into account not only the amount of space that A occupies but also the fill that occurs as R is computed. This all looks easy to the user, because MATLAB takes care of all of the details.

Exercise 1.6.3 A larger sparse matrix example supplied by MATLAB is the nonsymmetric 479×479 matrix `west0479`. To access this matrix type `load west0479`. Then type `A = west0479`; to get a simple abbreviation. Next type `size(A)` to confirm the dimensions and `issparse(A)` to find out whether A is stored as a sparse or a full matrix. The answer 1 indicates sparse and 0 indicates full. Type `spy(A)` to get a picture of the nonzero structure of A . Notice that it is nonsymmetric. (Type simply `A` to get a long list of the nonzero entries of A .) The spy plot tells you that `west0479` has 1887 nonzero entries. This information can also be obtained by typing `nnz(A)`.

Since `west0479` is not positive definite, or even symmetric, it does not have a Cholesky decomposition. To get a positive definite matrix, type `A = A' * A`; or `A = west0479' * west0479`; . Use commands such as `issparse`, `nnz`, and `spy` to find out whether your new A is being stored as a sparse matrix, how many nonzeros it has, and what its sparsity pattern looks like. Calculate the Cholesky factor of A and several reorderings of A :

⁵If more than one entry is assigned to a given (i, j) location, entries assigned to the same location are added together. For example, if sparse matrix A has $(i(3), j(3)) = (i(7), j(7)) = (i(50), j(50)) = (21, 36)$ (and all other (i, j) pairs differ from $(21, 36)$), then $a_{21,36} = s(3) + s(7) + s(50)$.

- (a) First consider A in its original ordering. Calculate the Cholesky factor of A , keeping track of how long it takes to do so. For example, you can use the commands

```
tic, R = chol(A); toc
```

or

```
t = cputime; R = chol(A); time = cputime - t
```

How many nonzeros does R have? Take a look at the spy plot of R .

- (b) Now permute the rows/columns of A randomly and repeat part (a). This is achieved as follows, for example:

```
p = randperm(479);
arnd = A(p,p);
spy(arnd)
tic, rrand = chol(arnd); toc
nz = nnz(rrand)
spy(rrand)
```

The first command returns a random permutation of the integers 1 through 479. The second produces a new matrix `arnd` whose rows and columns have been permuted according to the random permutation.

- (c) Repeat part (b) using the reverse Cuthill-McKee ordering instead of a random reordering. The correct permutation is obtained by replacing the random permutation by `p = symrcm(A);`. Thus

```
p = symrcm(A);
arcm = A(p,p);
spy(arcm)
tic, rrcm = chol(arcm); toc
nz = nnz(rrcm)
spy(rrcm)
```

The command `symrcm` is a mnemonic for “SYMmetric Reverse Cuthill-McKee.”

- (d) Repeat part (c) using the minimum-degree ordering instead of reverse Cuthill-McKee. The minimum-degree permutation is obtained by `p = symmnd(A);` `symmnd` is a mnemonic for “SYMmetric MiniMum Degree.”
- (e) Comment on your results.

□

Exercise 1.6.4 The MATLAB command `delsq` generates sparse matrices associated with discretized Laplacian (“del squared”) operators on various regions. For example, try

```

m = 32
A = delsq(numgrid('S',m));
issparse(A)
size(A)

```

This produces a 900×900 matrix. An advantage of this example is that it is scalable. If this matrix is too small or too big, a larger or smaller version can be obtained by increasing or decreasing m . In general the matrix A has dimension $(m - 2)^2$. Its structure is the same as that of the matrices discussed in Example 1.5.2 and Exercise 1.5.4. For more information on `delsq` type `help delsq` and `help numgrid` in MATLAB. Numerous variations can be obtained by replacing the 'S' by other letters in the `numgrid` command.

Using the matrix A generated as shown above (using a larger m if your computer allows it), calculate the Cholesky factor of A and several reorderings of A . Use the MATLAB commands that you learned in Exercise 1.6.3.

- (a) Make a spy plot of A . Notice that the original ordering already gives a narrow bandwidth. Calculate the Cholesky factor of A , noting the CPU time. How many nonzeros does the Cholesky factor have? Take a look at its spy plot.
- (b) Repeat part (a) using a random reordering of the rows/columns of A .
`(p = randperm((m-2)^2); arnd = a(p,p);).`
- (c) Repeat part (a) using the reverse Cuthill-McKee ordering.
- (d) Repeat part (a) using the minimum-degree ordering.
- (e) Another ordering that is available for this particular example is the *nested-dissection* ordering. Type `Anest = delsq(numgrid('N',m))`. This gives the same matrix as before, except that the rows/columns are numbered according to a nested-dissection ordering [30]. Repeat part (a) using the nested-dissection ordering.
- (f) Discuss the results from parts (a)–(e).

□

Exercise 1.6.5 Another interesting example that is supplied with MATLAB is the “NASA airfoil.” Run the NASA Airfoil command-line Demo by typing `airfoil` in MATLAB. This begins by plotting a grid of triangles that has been used for computation of the flow around an airplane wing by the finite element method. The grid has 4253 points or nodes, each having from three to nine neighbors. The Demo also builds a positive definite matrix A related to the adjacency matrix of the grid. This is a 4253×4253 matrix with a -1 in the (i, j) position if $i \neq j$ and node i is adjacent to node j (the nodes having been numbered beforehand). The main diagonal entries are made large enough that the matrix is positive definite. The exact recipe for A is given in the Demo. The Demo then proceeds to produce spy plots of A and several

reorderings of A . Once you have run the Demo, the matrix A remains in memory for your use.

- (a) Calculate the Cholesky factor of A , noting the CPU time. How many nonzeros does the Cholesky factor have? Take a look at its spy plot.
- (b) Repeat part (a) using the reverse Cuthill-McKee ordering.
- (c) Repeat part (a) using the minimum-degree ordering.
- (d) Discuss the results from parts (a)–(c).

□

MATLAB comes equipped with many other test matrices, both sparse and full. Type `help elmat` for a partial list. In particular, the `gallery` collection of N. J. Higham contains many good specimens. Type `help gallery` and follow the instructions.

Exercise 1.6.6 Repeat Exercise 1.6.4 (skipping part (e)) using a Wathen matrix from Higham's gallery. Type `A = gallery('wathen', 20, 15)`, for example. For a bigger matrix replace the 20 and 15 by larger numbers. Type `help private/wathen` for information about this matrix family. □

1.7 GAUSSIAN ELIMINATION AND THE LU DECOMPOSITION

In this and the next section, we will consider the problem of solving a system of n linear equations in n unknowns $Ax = b$ by Gaussian elimination. The algorithms developed here produce (in the absence of rounding errors) the unique solution whenever A is nonsingular. A is not assumed to have any special properties such as symmetry or positive definiteness.

Our strategy will be to transform the system $Ax = b$ to an equivalent system $Ux = y$ whose coefficient matrix is upper triangular. The system $Ux = y$ can then be solved easily by back substitution if U is nonsingular. To say that two systems are *equivalent* is to say that they have the same solutions.

We will transform the system by means of *elementary operations* of three types:

1. Add a multiple of one equation to another equation.
2. Interchange two equations.
3. Multiply an equation by a nonzero constant.

Proposition 1.7.1 *If $\hat{A}x = \hat{b}$ is obtained from $Ax = b$ by an elementary operation of type 1, 2, or 3, then the systems $Ax = b$ and $\hat{A}x = \hat{b}$ are equivalent.*

Exercise 1.7.2 Prove Proposition 1.7.1. *Discussion:* Suppose the system $Ax = b$ is transformed to $\hat{A}x = \hat{b}$ by an operation of type 1. You must show that (a) every solution

of $Ax = b$ is a solution of $\hat{A}x = \hat{b}$ and (b) every solution of $\hat{A}x = \hat{b}$ is a solution of $Ax = b$. Part (a) should be easy. Part (b) becomes easy when you realize that $Ax = b$ can be recovered from $\hat{A}x = \hat{b}$ by an operation of type 1: If $\hat{A}x = \hat{b}$ was obtained from $Ax = b$ by adding m times the j th row to the i th row, then $Ax = b$ can be recovered from $\hat{A}x = \hat{b}$ by adding $-m$ times the j th row to the i th row. Analogous remarks apply to operations of types 2 and 3. $\square$

It is convenient to represent the system $Ax = b$ by an augmented matrix $[A \mid b]$. Each equation in the system $Ax = b$ corresponds to a row of the matrix $[A \mid b]$. The elementary operations on the equations amount to the following *elementary row operations* on $[A \mid b]$:

1. Add a multiple of one row to another row.
2. Interchange two rows.
3. Multiply a row by a nonzero constant.

One can also apply elementary row operations to A alone, leaving off the vector b .

Proposition 1.7.3 *Suppose $\hat{A}$ is obtained from A by an elementary row operation of type 1, 2, or 3. Then $\hat{A}$ is nonsingular if and only if A is.*

Proof. By Theorem 1.2.3, A is nonsingular if and only if the system $Ay = 0$ has no solutions other than $y = 0$. The same holds for $\hat{A}$. If $\hat{A}$ is obtained from A by some elementary row operation, then the same row operation will transform the augmented matrix $[A \mid 0]$ to $[\hat{A} \mid 0]$, since the row operation (regardless of type) cannot create nonzero entries from the zeros in the last column. Thus, by Proposition 1.7.1, the systems $Ay = 0$ and $\hat{A}y = 0$ are equivalent, that is, they have the same solutions. Therefore $\hat{A}$ is nonsingular if and only if A is. $\square$

Elementary row operations are explored in more depth in Exercises 1.7.34 through 1.7.36. These exercises relate row operation to multiplication by *elementary matrices* of types 1, 2, and 3. They also contain an alternate proof of Proposition 1.7.3 that uses determinants.

In this section our focus will be on operations of type 1. In the next section, type 2 operations will come into play. We will not use type 3 operations, except in the discussion of scaling in Section 2.8.

Gaussian Elimination without Row Interchanges

We will approach the problem of solving $Ax = b$ in two stages. In the first stage, which will occupy us for the rest of this section, we will assume that A satisfies a special property that makes it possible to transform A to upper triangular form using elementary row operations of type 1 only. The algorithm that we will derive for carrying out this transformation is called *Gaussian elimination without row interchanges* or *Gaussian elimination without pivoting*.

Recall that for $k = 1, \dots, n$, the k th leading principal submatrix of A is the matrix A_k obtained by intersecting the first k rows and columns of A . For the rest of this section, the following assumption will hold:

$$A_k \text{ is nonsingular} \quad k = 1, \dots, n. \quad (1.7.4)$$

In particular, $A (= A_n)$ is itself nonsingular, so the system $Ax = b$ has a unique solution.

The reduction to triangular form is carried out in $n - 1$ steps. In the first step appropriate multiples of the first row are subtracted from each of the other rows to create zeros in positions $(2, 1), (3, 1), \dots, (n, 1)$. It is clear that in order to do this we must have $a_{11} \neq 0$. This is guaranteed by the assumption that $A_1 = [a_{11}]$ is nonsingular. The appropriate multiplier for the i th row is

$$m_{i1} = a_{i1}/a_{11} \quad i = 2, \dots, n. \quad (1.7.5)$$

Exercise 1.7.6 Verify that if m_{i1} times the first row is subtracted from the i th row of $[A \mid b]$, the resulting array has a zero in the $(i, 1)$ position. $\square$

Thus we carry out the operations

$$\begin{aligned} a_{ij}^{(1)} &= a_{ij} - m_{i1}a_{1j} & j = 2, \dots, n, \quad i = 2, \dots, n \\ b_i^{(1)} &= b_i - m_{i1}b_1 & i = 2, \dots, n \end{aligned} \quad (1.7.7)$$

to reduce $[A \mid b]$ to the form

$$\left[\begin{array}{c|ccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(1)} & b_n^{(1)} \end{array} \right].$$

It is not necessary to calculate the entries $a_{i1}^{(1)}$, $i = 2, \dots, n$, explicitly, because we know in advance that they are all zero.

In a computer implementation of this step, it is not necessary to store the zeros in the first column. Those storage spaces can be used for something else. As we shall soon see, it turns out to be a good idea to store the multipliers $m_{21}, m_{31}, \dots, m_{n1}$ there. Thus the array that initially contained A and b would look as follows after the first step:

$$\left[\begin{array}{c|ccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ m_{21} & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & \vdots & & \vdots & \vdots \\ m_{n1} & a_{n2}^{(2)} & \cdots & a_{nn}^{(1)} & b_n^{(1)} \end{array} \right].$$

The cost of the arithmetic in the first step is easily determined by examining (1.7.5) and (1.7.7). There are $n - 1$ divisions in (1.7.5) and $(n - 1)^2 + (n - 1)$ multiplications

and an equal number of additions in (1.7.7). The total flop count is $(2n + 1)(n - 1)$, which is approximately $2n^2$.

The second step operates on rows 2, 3, ..., n . Any operations that are performed on these rows will leave the zeros in column 1 undisturbed, because subtraction of a multiple of zero from zero leaves zero. Thus the second step ignores both the first row and the first column. Appropriate multiples of the second row are subtracted from rows 3, 4, ..., n to create zeros in positions $(3, 2)$, $(4, 2)$, ..., $(n, 2)$. Thus the second step is identical to the first, except that it operates on the submatrix

$$\left[\begin{array}{ccc|c} a_{22}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & & \vdots & \vdots \\ a_{n2}^{(1)} & \cdots & a_{nn}^{(1)} & b_n^{(1)} \end{array} \right].$$

The operations are

$$m_{i2} = a_{i2}^{(1)} / a_{22}^{(1)} \quad i = 3, \dots, n$$

and

$$\begin{aligned} a_{ij}^{(2)} &= a_{ij}^{(1)} - m_{i2} a_{2j}^{(1)} & j = 3, \dots, n, \quad i = 3, \dots, n \\ b_i^{(2)} &= b_i^{(1)} - m_{i2} b_2^{(1)} & i = 3, \dots, n. \end{aligned}$$

As in the first step, there is no need to calculate $a_{i2}^{(2)}$ explicitly for $i = 3, \dots, n$, because the multipliers m_{i2} were chosen so that $a_{i2}^{(2)} = 0$.

In order to carry out this step, we need $a_{22}^{(1)} \neq 0$. That this is so follows from the assumption that

$$A_2 = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

is nonsingular. After the first step, A_2 has been transformed to

$$\hat{A}_2 = \begin{bmatrix} a_{11} & a_{12} \\ 0 & a_{22}^{(1)} \end{bmatrix}.$$

$\hat{A}_2$ was obtained from A_2 by subtracting m_{21} times the first row from the second, a type 1 row operation. Therefore, by Proposition 1.7.3, the nonsingularity of A_2 implies the nonsingularity of $\hat{A}_2$. Since the $\hat{A}_2$ is upper triangular and has $a_{22}^{(1)}$ on its main diagonal, we conclude that $a_{22}^{(1)} \neq 0$, as claimed.

After the second step the augmented matrix will have been transformed to

$$\left[\begin{array}{cc|ccc|c} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} & b_1 \\ 0 & a_{22}^{(1)} & a_{23}^{(1)} & \cdots & a_{2n}^{(1)} & b_2^{(1)} \\ \hline 0 & 0 & a_{33}^{(2)} & \cdots & a_{3n}^{(2)} & b_3^{(2)} \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & a_{n3}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{array} \right].$$

In a computer implementation the zeros will be replaced by the multipliers $m_{21}, \dots, m_{n1}$ and $m_{32}, \dots, m_{n2}$. Since the second step is the same as the first, but on a matrix with one less row and one less column, the flop count for the second step is about $2(n-1)^2$.

The third step is identical to the previous two, except that it operates on the smaller matrix

$$\left[\begin{array}{ccc|c} a_{33}^{(2)} & \cdots & a_{3n}^{(2)} & b_3^{(2)} \\ \vdots & & \vdots & \vdots \\ a_{n3}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{array} \right].$$

In order to carry out the step, we need to have $a_{33}^{(2)} \neq 0$. This is guaranteed by the assumption that A_3 is nonsingular. After the first two steps, A_3 will have been transformed to

$$\hat{A}_3 = \left[\begin{array}{ccc} a_{11} & a_{12} & a_{13} \\ 0 & a_{22}^{(1)} & a_{23}^{(1)} \\ 0 & 0 & a_{33}^{(2)} \end{array} \right]$$

via two elementary row operations of type 1. Therefore, by Proposition 1.7.3, the nonsingularity of A_3 implies the nonsingularity of $\hat{A}_3$. This implies, in turn, that $a_{33}^{(2)} \neq 0$. The arithmetic for the third step amounts to about $2(n-2)^2$ flops.

After $n-1$ steps the system will be reduced to $[U | y]$, where U is upper triangular. For each k , the possibility of carrying out step k is guaranteed by the assumption that A_k is nonsingular. In the end we know that U is nonsingular because A is. Thus the system $Ux = y$ can be solved by back substitution to yield x , the unique solution of $Ax = b$.

The total flop count for the reduction to triangular form is approximately

$$2n^2 + 2(n-1)^2 + 2(n-2)^2 + \dots = 2 \sum_{k=1}^n k^2.$$

As in Section 1.4 (cf. Exercises 1.4.70 and 1.4.71) we can approximate this sum by an integral:

$$2 \sum_{k=1}^n k^2 \approx 2 \int_0^n k^2 dk = \frac{2}{3}n^3.$$

The additional cost of the back substitution is n^2 flops, which is relatively insignificant for large n . Thus the total cost of solving $Ax = b$ by this method is about $\frac{2}{3}n^3$ flops. This is about twice the cost of solving a positive definite system by Cholesky's method, which saves a factor of 2 by exploiting the symmetry of the coefficient matrix.

Example 1.7.8 Let

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 2 & 2 & -1 \\ 4 & -1 & 6 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 9 \\ 9 \\ 16 \end{bmatrix}.$$

Notice that $\det(A_1) = 2$, $\det(A_2) = 2$, and $\det(A_3) = -4$, so the leading principal submatrices are all nonsingular. This guarantees that we will be able to transform A to upper triangular form by row operations of type 1 only. It also guarantees that the system $Ax = b$ has a unique solution. In order to solve this system, we form the augmented matrix

$$[A \mid b] = \left[\begin{array}{ccc|c} 2 & 1 & 1 & 9 \\ 2 & 2 & -1 & 9 \\ 4 & -1 & 6 & 16 \end{array} \right].$$

The multipliers for the first step are $m_{21} = a_{21}/a_{11} = 1$ and $m_{31} = a_{31}/a_{11} = 2$. Thus we subtract 1 times the first row from the second row and 2 times the first row from the third row to create zeros in the (2, 1) and (3, 1) positions. Performing these row operations, we obtain

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 9 \\ 0 & 1 & -2 & 0 \\ 0 & -3 & 4 & -2 \end{array} \right].$$

The multiplier for the second step is $m_{32} = a_{32}^{(1)}/a_{22}^{(1)} = -3$. Thus we subtract -3 times the second row from the third row to obtain

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 9 \\ 0 & 1 & -2 & 0 \\ 0 & 0 & -2 & -2 \end{array} \right].$$

After two steps the reduction is complete. If we save the multipliers in place of the zeros, the array looks like this:

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 9 \\ \boxed{1} & \boxed{1} & -2 & 0 \\ 2 & -3 & -2 & -2 \end{array} \right]. \quad (1.7.9)$$

We can now solve the system by solving

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & -2 \\ 0 & 0 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 0 \\ -2 \end{bmatrix}$$

by back substitution. Doing so, we find that $x_3 = 1$, $x_2 = 2$, and $x_1 = 3$. $\square$

Exercise 1.7.10 Let

$$A = \begin{bmatrix} 2 & 1 & -1 & 3 \\ -2 & 0 & 0 & 0 \\ 4 & 1 & -2 & 6 \\ -6 & -1 & 2 & -3 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 13 \\ -2 \\ 24 \\ -14 \end{bmatrix}.$$

- (a) Calculate the appropriate (four) determinants to show that A can be transformed to (nonsingular) upper-triangular form by operations of type 1 only. (By the

way, this is strictly an academic exercise. In practice one never calculates these determinants in advance.)

- (b) Carry out the row operations of type 1 to transform the system $Ax = b$ to an equivalent system $Ux = y$, where U is upper triangular. Save the multipliers for use in Exercise 1.7.18.
- (c) Carry out the back substitution on the system $Ux = y$ to obtain the solution of $Ax = b$. Don't forget to check your work.

□

In real-life problems it will not be practical to verify in advance that the conditions (1.7.4) are satisfied. This does not of itself stop us from proceeding with the algorithm. In the course of performing the eliminations, if some A_k turns out to be singular (and all previous A_j were nonsingular), this fact will be signalled by $a_{kk}^{(k-1)} = 0$. This gives us, in principle, a test that we can apply in the course of transforming A to triangular form, to determine whether or not the conditions (1.7.4) are satisfied.

Although this test is perfectly satisfying in theory, it fails badly in practice. In real problems the entries are not all integers, the computations are done on a computer, and there are roundoff errors. Consequently an $a_{kk}^{(k-1)}$ that should be exactly zero in theory will typically turn out to be nonzero (albeit tiny) in practice. The test then fails to stop the computation, we divide by this erroneous number to obtain the multipliers for the k th step, and we end up with a disastrously inaccurate result. We will see how to deal with this problem in the next section.

Interpretation of the Multipliers

Now let us see why it is a good idea to save the multipliers m_{ij} . Suppose we have solved the system $Ax = b$, and we now wish to solve another system $Ax = \hat{b}$, which has the same coefficient matrix but a different right-hand side. We could form the augmented matrix $[A \mid \hat{b}]$ and solve the system from scratch, but this would be inefficient. Since the coefficient matrix is the same as before, all of the multipliers and row operations will be the same. If the multipliers have been saved, we can perform the row operations on the $\hat{b}$ column only and save a good many computations. Let's see how this works. The operations on b were as follows:

$$\begin{array}{lll}
 b_i^{(1)} & = & b_i - m_{i1}b_1 \quad i = 2, 3, \dots, n \\
 b_i^{(2)} & = & b_i^{(1)} - m_{i2}b_2^{(1)} \quad i = 3, 4, \dots, n \\
 \vdots & & \vdots \\
 b_i^{(j)} & = & b_i^{(j-1)} - m_{ij}b_j^{(j-1)} \quad i = j+1, \dots, n \\
 \vdots & & \vdots \\
 b_i^{(n-1)} & = & b_i^{(n-2)} - m_{i,n-1}b_{n-1}^{(n-2)} \quad i = n.
 \end{array} \tag{1.7.11}$$

At the end of the operations b has been transformed to

$$\begin{bmatrix} b_1 \\ b_2^{(1)} \\ b_3^{(2)} \\ \vdots \\ b_n^{(n-1)} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = y. \quad (1.7.12)$$

The same operations applied to $\hat{b}$ would yield a vector $\hat{y}$. The system $Ax = \hat{b}$ could then be solved by solving the equivalent upper-triangular system $Ux = \hat{y}$, where U is the upper-triangular matrix that was obtained in the original reduction of A .

Example 1.7.13 Suppose we wish to solve the system $Ax = \hat{b}$, where

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 2 & 2 & -1 \\ 4 & -1 & 6 \end{bmatrix} \quad \text{and} \quad \hat{b} = \begin{bmatrix} 7 \\ 3 \\ 20 \end{bmatrix}.$$

The coefficient matrix is the same as in Example 1.7.8, so the multipliers are as shown in (1.7.9), that is, $m_{21} = 1$, $m_{31} = 2$, and $m_{32} = -3$. This means that A can be transformed to upper-triangular form by subtracting 1 times the first row from the third row, 2 times the first row from the third row, and -3 times the (new) second row from the third row. Rather than performing these operations on the augmented matrix $[A \mid \hat{b}]$, we apply them to the column $\hat{b}$ only and get

$$\begin{aligned} \hat{b}_2^{(1)} &= \hat{b}_2 - m_{21}\hat{b}_1 = 3 - 1 \cdot 7 = -4 \\ \hat{b}_3^{(1)} &= \hat{b}_3 - m_{31}\hat{b}_1 = 20 - 2 \cdot 7 = 6 \\ \hat{b}_3^{(2)} &= \hat{b}_3^{(1)} - m_{32}\hat{b}_2^{(1)} = 6 - (-3)(-4) = -6. \end{aligned}$$

After these transformations, the new right-hand side is

$$\hat{y} = \begin{bmatrix} b_1 \\ b_2^{(1)} \\ b_3^{(2)} \end{bmatrix} = \begin{bmatrix} 7 \\ -4 \\ -6 \end{bmatrix}.$$

Now we can get the solution by solving $Ux = y$ by back substitution, where

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & -2 \\ 0 & 0 & -2 \end{bmatrix},$$

as in Example 1.7.8. Doing so, we find that $x_3 = 3$, $x_2 = 2$, and $x_1 = 1$. □

A quick count shows that (1.7.11) takes about n^2 flops. Adding this to the n^2 flops for back substitution, we find that the total arithmetic cost of solving $Ax = \hat{b}$

is about $2n^2$ flops. When we compare this with the $\frac{2}{3}n^3$ flops that were needed to transform A to upper triangular form initially, we conclude that once we have solved $Ax = b$, we can solve additional systems with the same coefficient matrix at little additional cost.

A closer look at the transformation of b to y yields an important interpretation of Gaussian elimination. By (1.7.12), we can rewrite (1.7.11) in terms of the components of y as follows:

$$\begin{aligned} b_i^{(1)} &= b_i - m_{i1}y_1 & i &= 2, 3, \dots, n \\ b_i^{(2)} &= b_i^{(1)} - m_{i2}y_2 & i &= 3, 4, \dots, n \\ \vdots & & \vdots & \\ b_i^{(n-1)} &= b_i^{(n-2)} - m_{i,n-1}y_{n-1} & i &= n. \end{aligned} \quad (1.7.14)$$

Together with (1.7.12), these equations can be used to derive expressions for $y_1, y_2, \dots, y_n$. By the first equation of (1.7.14), $y_2 = b_2^{(1)} = b_2 - m_{21}y_1$. By the first two equations of (1.7.14), $y_3 = b_3^{(2)} = b_3^{(1)} - m_{32}y_2 = b_3 - m_{31}y_1 - m_{32}y_2$. Similarly $y_4 = b_4 - m_{41}y_1 - m_{42}y_2 - m_{43}y_3$, and in general

$$y_i = b_i - \sum_{j=1}^{i-1} m_{ij}y_j \quad i = 1, 2, \dots, n. \quad (1.7.15)$$

We can use (1.7.15) to calculate $y_1, y_2, \dots, y_n$. Clearly the operations are the same as those of (1.7.11), but in a different order. The equations (1.7.15) can be interpreted as a matrix operation if we rewrite them as

$$\sum_{j=1}^{i-1} m_{ij}y_j + y_i = b_i, \quad i = 1, \dots, n.$$

This can be expressed as the matrix equation

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ m_{21} & 1 & 0 & \ddots & \vdots \\ m_{31} & m_{32} & 1 & \ddots & \\ \vdots & & & \ddots & 0 \\ m_{n1} & m_{n2} & m_{n3} & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix}. \quad (1.7.16)$$

Thus we see that y is just the solution of a linear system $Ly = b$, where L is lower triangular. In fact L is *unit lower triangular*, which means that its main diagonal entries are all ones. We learned in Section 1.3 that any lower-triangular system can be solved by forward substitution. In fact (1.7.15) is just row-oriented forward substitution. The divisions that are generally required (as in (1.3.3)) are absent from (1.7.15) because the main-diagonal entries of (1.7.16) are ones. You can easily check that (1.7.11) is nothing but column-oriented forward substitution.

A brief summary of what we have done so far will lead to an interesting and important conclusion: L and U can be interpreted as factors of A . In order to solve the system

$$Ax = b,$$

we reduced it to the form

$$Ux = y,$$

where U is upper triangular, and y is the solution of a unit lower-triangular system

$$Ly = b.$$

Combining these last two equations, we find that $LUx = b$. Thus $LUx = b = Ax$. These equations hold for any choice of b , and hence for any choice of x . (For a given x , the appropriate b is obtained by the calculation $b = Ax$.) Since the equation $LUx = Ax$ holds for all $x \in \mathbb{R}^n$, it must be that

$$A = LU.$$

We conclude that Gaussian elimination without row interchanges (saving the multipliers) can be viewed as a process of decomposing A into a product, $A = LU$, where L is lower triangular and U is upper triangular. In fact the usual procedure is not to form an augmented matrix $[A \mid b]$ but to do row operations on A alone. A is reduced, saving multipliers, to the form

$$\left[\begin{array}{ccccc} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ m_{21} & u_{22} & u_{23} & \cdots & u_{2n} \\ m_{31} & m_{32} & u_{33} & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ m_{n1} & m_{n2} & m_{n3} & & u_{nn} \end{array} \right],$$

which contains all information about L and U . The system $LUx = b$ is then solved by first solving $Ly = b$ for y by forward substitution and then solving $Ux = y$ by back substitution.

Example 1.7.17 Solve the system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 2 & 2 & -1 \\ 4 & -1 & 6 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 3 \\ 0 \\ 11 \end{bmatrix}.$$

The coefficient matrix is the same as in Example 1.7.8. From (1.7.9) we know that $A = LU$, where

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & -3 & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & -2 \\ 0 & 0 & -2 \end{bmatrix}.$$

Solving $Ly = b$ by forward substitution, we get $y = [3, -3, -4]^T$. Solving $Ux = y$ by back substitution, we get $x = [0, 1, 2]^T$. $\square$

Exercise 1.7.18 Solve the linear system $Ax = \hat{b}$, where A is as in Exercise 1.7.10 and $\hat{b} = [12, -8, 21, -26]^T$. Use the L and U that you calculated in Exercise 1.7.10. $\square$

We have already proved most of the following theorem.

Theorem 1.7.19 (*LU Decomposition Theorem*) Let A be an $n \times n$ matrix whose leading principal submatrices are all nonsingular. Then A can be decomposed in exactly one way into a product

$$A = LU,$$

such that L is unit lower triangular and U is upper triangular.

Proof. We have already shown that L and U exist.⁶ It remains only to show that they are unique. Our uniqueness proof will yield a second algorithm for calculating the LU decomposition. Look at the equation $A = LU$ in detail.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ l_{21} & 1 & 0 & \cdots & 0 \\ l_{31} & l_{32} & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ 0 & u_{22} & u_{23} & \cdots & u_{2n} \\ 0 & 0 & u_{33} & \cdots & u_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & u_{nn} \end{bmatrix}.$$

The first row of L is known completely, and it has only one nonzero entry. Multiplying the first row of L by the j th column of U , we find that

$$a_{1j} = 1u_{1j} + 0u_{2j} + 0u_{3j} + \cdots + 0u_{nj}.$$

That is, $u_{1j} = a_{1j}$. Thus the first row of U is uniquely determined. Now that we know the first row of U , we see that the first column of U is also known, since its only nonzero entry is u_{11} . Multiplying the i th row of L by the first column of U , we find that

$$a_{i1} = l_{i1}u_{11}, \quad i = 2, \dots, n. \quad (1.7.20)$$

The assumption that A is nonsingular implies that U is also nonsingular. (Why?) Hence $u_{kk} \neq 0$ for $k = 1, \dots, n$, and, in particular, $u_{11} \neq 0$. Therefore (1.7.20) determines the first column of L uniquely:

$$l_{i1} = \frac{a_{i1}}{u_{11}}, \quad i = 2, \dots, n.$$

⁶Exercise 1.7.47 outlines a second existence proof.

Thus the first column of L is uniquely determined. Now that the first row of U and first column of L have been determined, it is not hard to show that the second row of U is also uniquely determined. As an exercise, determine a formula for u_{2j} ($j \geq 2$) in terms of a_{2j} and entries of the first row of U and column of L . Once u_{22} is known, it is possible to determine the second column of L . Do this also as an exercise.

Now suppose the first $k - 1$ rows of U and columns of L have been shown to be uniquely determined. We will show that the k th row of U and column of L are uniquely determined; this will prove uniqueness by induction. The k th row of L is $[l_{k1} \ l_{k2} \ \cdots \ l_{k,k-1} \ 1 \ 0 \ \cdots \ 0]$. Since $l_{k1}, \dots, l_{k,k-1}$ are all in the first $k - 1$ columns of L , they are uniquely determined. Multiplying the k th row of L by the j th column of U ($j \geq k$), we find that

$$a_{kj} = \sum_{m=1}^{k-1} l_{km}u_{mj} + u_{kj}. \quad (1.7.21)$$

All of the u_{mj} (aside from u_{kj}) lie in the first $k - 1$ rows of U and are therefore known (i.e., uniquely determined). Therefore u_{kj} is uniquely determined by (1.7.21):

$$u_{kj} = a_{kj} - \sum_{m=1}^{k-1} l_{km}u_{mj}, \quad j = k, k+1, \dots, n. \quad (1.7.22)$$

This proves that the k th row of U is uniquely determined and provides a way of computing it. Now that u_{kk} is known, the entire k th column of U is determined. Multiplying the i th row of L by the k th column of U , we find that

$$a_{ik} = \sum_{m=1}^{k-1} l_{im}u_{mk} + l_{ik}u_{kk}. \quad (1.7.23)$$

All of the l_{im} (aside from l_{ik}) lie in the first $k - 1$ columns of L and are therefore uniquely determined. Furthermore $u_{kk} \neq 0$. Thus (1.7.23) determines l_{ik} uniquely:

$$l_{ik} = u_{kk}^{-1} \left(a_{ik} - \sum_{m=1}^{k-1} l_{im}u_{mk} \right), \quad i = k+1, \dots, n. \quad (1.7.24)$$

This proves that the k th column of L is uniquely determined and provides a way of calculating it. The proof that L and U are unique is now complete. $\square$

Equations (1.7.22) and (1.7.24), applied in the correct order, provide a means of calculating L and U . Because both (1.7.22) and (1.7.24) require inner-product computations, we will call this algorithm the *inner-product formulation* of Gaussian elimination.⁷ Historically it has been known as the *Doolittle reduction*.⁸

⁷In contrast, the calculation of L and U by row operations of type 1 is an *outer product* formulation. See Exercise 1.7.27.

⁸A well-known variant is the *Crout reduction*, which is very similar but produces a decomposition in which U , instead of L , has ones on the main diagonal.

In the calculation of L and U by row operations, the entries of A are gradually replaced by entries of L and U . The same can be done if the inner-product formulation is used: a_{kj} ($k \leq j$) is used only to compute u_{kj} , and a_{ik} ($i > k$) is used only to compute l_{ik} . Therefore each entry of L or U can be stored in place of the corresponding entry of A as soon as it is computed. You should convince yourself that the two methods perform exactly the same operations, although not in the same order. In Gaussian elimination by row operations a typical entry is modified numerous times before the final result is obtained. In the inner-product formulation the same modifications are made, but all of the modifications of each entry are made at once.

Example 1.7.25 Let

$$A = \begin{bmatrix} 2 & 4 & 2 & 3 \\ -2 & -5 & -3 & -2 \\ 4 & 7 & 6 & 8 \\ 6 & 10 & 1 & 12 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} -3 \\ 3 \\ -1 \\ -16 \end{bmatrix}.$$

We will calculate L and U such that $A = LU$ by two different methods. First let's do Gaussian elimination by row operations.

Step 1:

$$\left[\begin{array}{cccc|c} 2 & 4 & 2 & 3 & \\ -1 & -1 & -1 & 1 & \\ 2 & -1 & 2 & 2 & \\ 3 & -2 & -5 & 3 & \end{array} \right]$$

Step 2:

$$\left[\begin{array}{cccc|c} 2 & 4 & 2 & 3 & \\ -1 & -1 & -1 & 1 & \\ 2 & -1 & 2 & 2 & \\ 3 & -2 & -5 & 3 & \end{array} \right]$$

Step 3:

$$\left[\begin{array}{cccc|c} 2 & 4 & 2 & 3 & \\ -1 & -1 & -1 & 1 & \\ 2 & -1 & 2 & 2 & \\ 3 & -2 & -5 & 3 & \end{array} \right]$$

Now Let's try the inner-product formulation.

Step 1:

$$\left[\begin{array}{cccc|c} 2 & 4 & 2 & 3 & \\ -1 & -5 & -3 & -2 & \\ 2 & 7 & 6 & 8 & \\ 3 & 10 & 1 & 12 & \end{array} \right]$$

The first row of U and column of L have been calculated. The rest of the matrix remains untouched.

Step 2:

$$\left[\begin{array}{cccc|cc} 2 & 4 & 2 & 3 & & \\ -1 & & & & & \\ \hline 2 & 1 & & & 6 & 8 \\ 3 & 2 & & & 1 & 12 \end{array} \right]$$

Step 3:

$$\left[\begin{array}{cccc|cc} 2 & 4 & 2 & 3 & & \\ -1 & & & & & \\ \hline 2 & 1 & & & 3 & 1 \\ 3 & 2 & -1 & & 12 & \end{array} \right]$$

Now only u_{44} remains to be calculated.

Step 4:

$$\left[\begin{array}{cccc|cc} 2 & 4 & 2 & 3 & & \\ -1 & & & & & \\ \hline 2 & 1 & & & 3 & 1 \\ 3 & 2 & -1 & & 2 & \end{array} \right]$$

Both reductions yield the same result. You might find it instructive to try the inner-product reduction by the erasure method. Begin with the entries of A entered in pencil. As you calculate each entry of L or U , erase the corresponding entry of A and replace it with the new result. Do the arithmetic in your head.

Now that we have the LU decomposition of A , we perform forward substitution on

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 2 & 1 & 1 & 0 \\ 3 & 2 & -1 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} -3 \\ 3 \\ -1 \\ 16 \end{bmatrix}$$

to get $y = [-3, 0, 5, -2]^T$. We then perform back substitution on

$$\begin{bmatrix} 2 & 4 & 2 & 3 \\ 0 & -1 & -1 & 1 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -3 \\ 0 \\ 5 \\ -2 \end{bmatrix}$$

to get $x = [4, -3, 2, -1]^T$. □

Exercise 1.7.26 Use the inner-product formulation to calculate the LU decomposition of the matrix A in Exercise 1.7.10 □

Exercise 1.7.27 Develop an outer-product formulation of the LU decomposition algorithm in the spirit of the outer-product formulation of the Cholesky decomposition algorithm. Show that this algorithm is identical to Gaussian elimination by row operations of type 1. □

Variants of the LU Decomposition

An important variant of the LU decomposition is the LDV decomposition, which has a diagonal matrix sandwiched between two unit-triangular matrices. A matrix B is a *diagonal* matrix if $b_{ij} = 0$ whenever $i \neq j$. Thus all of the entries off of the main diagonal are zero.

Theorem 1.7.28 (*LDV Decomposition Theorem*) *Let A be an $n \times n$ matrix whose leading principal submatrices are all nonsingular. Then A can be decomposed in exactly one way as a product*

$$A = LDV,$$

such that L is unit lower triangular, D is diagonal, and V is unit upper triangular.

Proof. By Theorem 1.7.19 there exist unit lower-triangular L and upper-triangular U such that $A = LU$. Since U is nonsingular, $u_{kk} \neq 0$ for $k = 1, \dots, n$. Let D be the diagonal matrix whose main-diagonal entries are $u_{11}, \dots, u_{nn}$. Then D is nonsingular; D^{-1} is the diagonal matrix whose main-diagonal entries are $u_{11}^{-1}, \dots, u_{nn}^{-1}$. Let $V = D^{-1}U$. You can easily check that V is unit upper triangular, and $A = LDV$.

To complete the proof, we must show that the decomposition is unique. Suppose $A = L_1 D_1 V_1 = L_2 D_2 V_2$. Let $U_1 = D_1 V_1$ and $U_2 = D_2 V_2$. Then obviously U_1 and U_2 are upper triangular, and $A = L_1 U_1 = L_2 U_2$. By the uniqueness of LU decompositions, $L_1 = L_2$ and $U_1 = U_2$. The latter equation implies $D_1 V_1 = D_2 V_2$; therefore

$$D_2^{-1} D_1 = V_2 V_1^{-1}. \quad (1.7.29)$$

Since V_1 is unit upper triangular, so is V_1^{-1} . Since V_2 and V_1^{-1} are unit upper triangular, so is $V_2 V_1^{-1}$. (See Exercises 1.7.44, 1.7.45, and 1.7.46.) On the other hand, $D_2^{-1} D_1$ is clearly diagonal. Thus by (1.7.29) $V_2 V_1^{-1}$ is both unit upper triangular and diagonal; that is, $V_2 V_1^{-1} = I$. Therefore $V_2 = V_1$ and $D_2 = D_1$. $\square$

Because of the symmetric roles played by L and V , the LDV decomposition is of special interest when A is symmetric.

Theorem 1.7.30 *Let A be a symmetric matrix whose leading principal submatrices are nonsingular. Then A can be expressed in exactly one way as a product $A = LDL^T$, such that L is unit lower triangular and D is diagonal.*

Proof. A has an LDV decomposition: $A = LDV$. We need only show that $V = L^T$. Now $A = A^T = (LDV)^T = V^T D^T L^T$. V^T is unit lower triangular, D^T is diagonal, and L^T is unit upper triangular, so $V^T D^T L^T$ is an LDV decomposition of A . By uniqueness of the LDV decomposition, $V = L^T$. $\square$

In Section 1.4 we proved that if A is positive definite, then each of the leading principal submatrices A_k is positive definite and hence nonsingular. Therefore every positive definite matrix satisfies the hypotheses of Theorem 1.7.30.

Theorem 1.7.31 *Let A be positive definite. Then A can be expressed in exactly one way as a product $A = LDL^T$, such that L is unit lower triangular, and D is a diagonal matrix whose main-diagonal entries are positive.*

Exercise 1.7.32 Prove Theorem 1.7.31. To this end it suffices to show that A is positive definite if and only if the main-diagonal entries of D are positive. See Proposition 1.4.55. $\square$

Theorem 1.7.31 leads to a second proof of the existence of the Cholesky decomposition: Since the main-diagonal entries of D are positive, we can take their square roots. Let $E = D^{1/2}$. This is the diagonal matrix whose main diagonal entries are $+\sqrt{d_{11}}, \dots, +\sqrt{d_{nn}}$. Then $A = LDL^T = LE^2L^T = LEE^TL^T = (LE)(LE)^T = R^TR$, where $R = (LE)^T$. Then R is upper triangular with positive main-diagonal entries, and $A = R^TR$.

The LDL^T decomposition is sometimes used in place of the Cholesky decomposition to solve positive definite systems. Algorithms analogous to the various formulations of Cholesky's method exist (Exercise 1.7.40). Each of these algorithms has block versions. The LDL^T decomposition is sometimes preferred because it does not require the extraction of square roots.

The next theorem presents a third decomposition for positive definite matrices that is preferred in some situations.

Theorem 1.7.33 *Let A be positive definite. Then A can be expressed in exactly one way as a product $A = MD^{-1}M^T$, such that M is lower triangular, D is a diagonal matrix whose main-diagonal entries are positive, and the main diagonal entries of M are the same as those of D .*

Proof. Starting with the decomposition $A = LDL^T$, let $M = LD$. Then clearly $A = MD^{-1}M^T$, and the main-diagonal entries of M are the same as those of D . The proof of uniqueness is left as an exercise. $\square$

Algorithms for calculating the $MD^{-1}M^T$ decomposition can be derived by making slight modifications to algorithms for the Cholesky or LDL^T decomposition. Like the LDL^T algorithms, the $MD^{-1}M^T$ algorithms do not require the computation of square roots. In a sense the $MD^{-1}M^T$ decomposition is more fundamental than the others; see the discussion of generic Gaussian elimination in Exercise 1.7.55.

The LDL^T and $MD^{-1}M^T$ algorithms can be applied to symmetric matrices that are not positive definite, provided the leading principal submatrices are not singular. However, it is difficult to check in advance whether these conditions are satisfied, and even if they are, the computation may be spoiled by roundoff errors. Stable, efficient algorithms for symmetric, indefinite linear systems do exist, but we will not discuss them here. See the discussion in [33, § 4.4].

Additional Exercises

Exercise 1.7.34 In this exercise you will show that performing an elementary row operation of type 1 is equivalent to left multiplication by a matrix of a special type. Suppose $\hat{A}$ is obtained from A by adding m times the j th row to the i th row.

- (a) Show that $\hat{A} = MA$, where M is the triangular matrix obtained from the identity matrix by replacing the zero by an m in the (i, j) position. For example, when $i > j$, M has the form

$$\begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \ddots & & \\ & m & & & 1 & \\ & & & & & \ddots \\ & & & & & & 1 \end{bmatrix}.$$

Notice that this is the matrix obtained by applying the type 1 row operation directly to the identity matrix. We call M an *elementary matrix* of type 1.

- (b) Show that $\det(M) = 1$ and $\det(\hat{A}) = \det(A)$. Thus we see (again) that $\hat{A}$ is nonsingular if and only if A is.
- (c) Show that M^{-1} differs from M only in that it has $-m$ instead of m in the (i, j) position. M^{-1} is also an elementary matrix of type 1. To which elementary operation does it correspond?

□

Exercise 1.7.35 Suppose $\hat{A}$ is obtained from A by interchanging the i th and j th rows. This is an operation of type 2.

- (a) Show that $\hat{A} = MA$, where M is the matrix obtained from the identity matrix by interchanging the i th and j th rows. We call this M an *elementary matrix* of type 2.
- (b) Show that $\det(M) = -1$ and $\det(\hat{A}) = -\det(A)$. Thus we see (again) that $\hat{A}$ is nonsingular if and only if A is.
- (c) Show that $M^{-1} = M$. Explain why this result would have been expected, in view of the action of M as the effector of an elementary operation.

□

Exercise 1.7.36 Suppose $\hat{A}$ is obtained from A by multiplying the i th row by the nonzero constant c .

- (a) Find the form of the matrix M (an *elementary matrix* of type 3) such that $\hat{A} = MA$.
- (b) Find M^{-1} and state its function as an elementary matrix.
- (c) Find $\det(M)$ and determine the relationship between $\det(\hat{A})$ and $\det(A)$. Deduce that $\hat{A}$ is nonsingular if and only if A is.

□

Exercise 1.7.37 Let A be a symmetric matrix with $a_{11} \neq 0$. Suppose A has been reduced to the form

$$\left[\begin{array}{c|cccc} a_{11} & a_{12} & \cdots & a_{1n} \\ \hline 0 & & & \\ \vdots & & A^{(1)} & \\ 0 & & & \end{array} \right]$$

by row operations of type 1 only. Prove that $A^{(1)}$ is symmetric. (It follows that by exploiting symmetry we can cut the arithmetic almost in half, provided A_k is nonsingular for $k = 1, \dots, n$. This is the case, e.g., if A is positive definite.) □

Exercise 1.7.38 (a) Develop a bordered form of Gaussian elimination analogous to the bordered form of the Cholesky decomposition algorithm. (b) Suppose A is sparse, its lower part is stored in a row-oriented envelope, and its upper part is stored in a column-oriented envelope. Prove that the envelope of L (by rows!) equals the envelope of the lower part of A , and the envelope of U (by columns) equals the envelope of the upper part of A . □

Exercise 1.7.39 We have seen that if the leading principal submatrices A_k are nonsingular for $k = 1, \dots, n$, then A has an LU decomposition. Prove the following converse. If A is nonsingular and has an LU decomposition, then A_k is nonsingular for $k = 1, \dots, n$. (Hint: Partition the matrices in the decomposition $A = LU$.) □

Exercise 1.7.40 Develop algorithms to calculate the LDL^T decomposition (Theorem 1.7.31) of a positive definite matrix: (a) inner-product formulation, (b) outer-product formulation, (c) bordered formulation. (d) Count the operations for each algorithm. You may find that $\frac{2}{3}n^3$ multiplications are required, twice as many for Cholesky's method. In this case, show how half of the multiplications can be moved out of the inner loop to cut the flop count to $n^3/3$. A small amount of extra storage space is needed to store the intermediate results. (e) Which of the three formulations is the same as the one suggested by Exercise 1.7.37? □

Exercise 1.7.41 Develop algorithms to calculate the $MD^{-1}M^T$ decomposition of a positive definite matrix (Theorem 1.7.33): (a) inner-product formulation, (b) outer-product formulation, (c) bordered formulation. Again the flop count is about $n^3/3$ if the algorithms are written carefully. □

Exercise 1.7.42 Prove the uniqueness part of Theorem 1.7.33. □

Exercise 1.7.43 Let A be a nonsymmetric matrix. (a) Prove that if the leading principal submatrices of A are all nonsingular, then there exist unique matrices M , D , and U , such that M is lower triangular, D is diagonal, U is upper triangular, M , D , and U all have the same entries on the main diagonal, and $A = MD^{-1}U$. (b) Derive an

algorithm that computes M , D , and U and stores them over A . (Notice that the array that holds A *does* have enough room to store M , D , and U .) $\square$

Exercise 1.7.44 Let L be a nonsingular, lower-triangular matrix.

- (a) Prove that L^{-1} is lower triangular. (*Outline:* Use induction on n , the dimension of L . Let $M = L^{-1}$. Partition L and M in the same way:

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}, \quad M = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix},$$

where L_{11} and M_{11} are square, and L_{11} and L_{22} are lower triangular and nonsingular. Since $M = L^{-1}$, we have $LM = I$. Partition this equation, and use it to deduce that $M_{12} = 0$, $M_{11} = L_{11}^{-1}$, and $M_{22} = L_{22}^{-1}$. Use the induction hypothesis to conclude that M_{11} and M_{22} are lower triangular. This is just one of numerous ways to organize this proof.)

- (b) Prove that the entries on the main diagonal of L^{-1} are $l_{11}^{-1}, l_{22}^{-1}, \dots, l_{nn}^{-1}$. Thus L^{-1} is unit lower triangular if L is. $\square$

Exercise 1.7.45 Let L and M be lower-triangular matrices.

- (a) Prove that LM is lower triangular.
- (b) Prove that the entries on the main diagonal of LM are $l_{11}m_{11}, \dots, l_{nn}m_{nn}$. Thus the product of two unit lower-triangular matrices is unit lower triangular. $\square$

Exercise 1.7.46 Prove the upper-triangular analogues of the results in Exercises 1.7.44 and 1.7.45. (The easy way to do this is to take transposes and invoke the lower-triangular results.) $\square$

Exercise 1.7.47 In this exercise you will extend Exercise 1.7.34, expressing the entire reduction to triangular form by type 1 operations in the language of matrices. Let A be a (nonsingular) matrix whose leading principal submatrices are all nonsingular.

- (a) Define a matrix M_1 by

$$M_1 = \begin{bmatrix} 1 & 0 \\ m_1 & I_{n-1} \end{bmatrix},$$

where I_{n-1} denotes the $(n-1) \times (n-1)$ identity matrix, and $m_1 = [m_{21}, \dots, m_{n1}]^T$. Show that

$$M_1^{-1} = \begin{bmatrix} 1 & 0 \\ -m_1 & I_{n-1} \end{bmatrix}.$$

- (b) The first step of Gaussian elimination without row interchanges transforms A to

$$A^{(1)} = \left[\begin{array}{c|ccc} a_{11} & a_{12} & \cdots & a_{1n} \\ \hline 0 & a_{22}^{(1)} & \cdots & a_{2n}^{(1)} \\ \vdots & \vdots & & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(1)} \end{array} \right].$$

Prove that $A^{(1)} = M_1^{-1}A$, where M_1 is as in part (a), and $m_{21}, \dots, m_{n1}$ are the multipliers for step 1, i.e. $m_{i1} = a_{i1}/a_{11}$.

- (c) Let $A^{(k)}$ denote the matrix to which A has been transformed after k steps of Gaussian elimination. Show that $A^{(k)} = M_k^{-1}A^{(k-1)}$, where

$$M_k = \left[\begin{array}{ccc} I_{k-1} & & \\ & 1 & 0 \\ & m_k & I_{n-k} \end{array} \right].$$

The blank areas in the array are zeros, and $m_k = [m_{k+1,k}, \dots, m_{nk}]^T$, where $m_{k+1,k}, \dots, m_{nk}$ are the multipliers for the k th step.

Matrices of the form of M_k and M_k^{-1} are sometimes called *Gauss transforms*, because they effect the transformations of Gaussian elimination.

- (d) After $n - 1$ steps of Gaussian elimination, A has been transformed to the upper-triangular matrix $A^{(n-1)} = U$. Prove that $A = LU$, where $L = M_1 M_2 \cdots M_{n-1}$. By Exercise 1.7.45 L is unit lower triangular.
- (e) Prove that the matrix $L = M_1 \cdots M_{n-1}$ from part (d) has the form

$$L = \left[\begin{array}{cccc} 1 & 0 & 0 & \cdots & 0 \\ m_{21} & 1 & 0 & \cdots & 0 \\ m_{31} & m_{32} & 1 & & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ m_{n1} & m_{n2} & m_{n3} & \cdots & 1 \end{array} \right].$$

For example, verify that

$$\begin{bmatrix} 1 & 0 \\ m & I_j \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \hat{M} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ m & \hat{M} \end{bmatrix},$$

then use this as the basis of an induction proof.

□

Exercise 1.7.48 Write down a careful proof of the existence of the LU decomposition, based on parts (a) through (d) of Exercise 1.7.47.

□

Exercise 1.7.49 Consider the Gauss transforms M_i introduced in Exercise 1.7.47. Show that each such matrix has the form $M = I - vw^T$, where v and w are appropriately chosen vectors. Since vw^T is a matrix of rank 1, matrices of the form $I - vw^T$ are called *rank-one updates of the identity matrix*. $\square$

Exercise 1.7.50 This exercise introduces the idea of *block* Gaussian elimination. Let A be a (nonsingular) matrix whose leading principal submatrices are all nonsingular. Partition A as

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where A_{11} is, say, $k \times k$. Since A_{11} is a leading principal submatrix, it is nonsingular.

(a) Show that there is exactly one matrix M such that

$$\begin{bmatrix} I_k & 0 \\ -M & I_{n-k} \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ 0 & \tilde{A}_{22} \end{bmatrix}.$$

In this equation we place no restriction on the form of $\tilde{A}_{22}$. The point is that we seek a transformation that makes the (2,1) block zero. This is a block Gaussian elimination operation; M is a block multiplier.

Show that the unique M that works is given by $M = A_{21}A_{11}^{-1}$, and this implies that

$$\tilde{A}_{22} = A_{22} - A_{21}A_{11}^{-1}A_{12}. \quad (1.7.51)$$

The matrix $\tilde{A}_{22}$ is called the *Schur complement* of A_{11} in A .

(b) Show that

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} I & 0 \\ M & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ 0 & \tilde{A}_{22} \end{bmatrix}. \quad (1.7.52)$$

This is a *block LU decomposition*.

(c) The leading principal submatrices of A_{11} are, of course, all nonsingular. Prove that $\tilde{A}_{22}$ is nonsingular. More generally, prove that all of the leading principal submatrices of $\tilde{A}_{22}$ are nonsingular.

(d) By part (c), both A_{11} and $\tilde{A}_{22}$ have *LU* decompositions, say $A_{11} = L_1U_1$ and $\tilde{A}_{22} = L_2U_2$. Show that

$$A = \begin{bmatrix} L_1 & 0 \\ ML_1 & L_2 \end{bmatrix} \begin{bmatrix} U_1 & L_1^{-1}A_{12} \\ 0 & U_2 \end{bmatrix}.$$

This is the *LU* decomposition of A .

(e) After k steps of Gaussian elimination without row interchanges, as discussed in this section and in Exercise 1.7.47, A has been transformed to

$$A^{(k)} = \begin{bmatrix} A_{11}^{(k)} & A_{12}^{(k)} \\ 0 & A_{22}^{(k)} \end{bmatrix},$$

where $A_{11}^{(k)}$ is upper triangular. The subsequent Gaussian elimination operations act on $A_{22}^{(k)}$ only, ignoring the rest of the matrix. Prove that

- (i) $A_{11}^{(k)} = U_1$, the upper-triangular factor of A_{11} ,
- (ii) $A_{22}^{(k)} = \tilde{A}_{22}$, the Schur complement of A_{11} in A .

□

Exercise 1.7.53 Write a careful induction proof of existence and uniqueness of the LU decomposition, based on the developments in parts (a) through (d) of the previous exercise. □

Exercise 1.7.54 (a) Prove that if H is nonsingular and symmetric, then H^{-1} is symmetric. (b) Prove that the Schur complement $\hat{A}_{22}$ (1.7.51) is symmetric if A is. This result generalizes Exercise 1.7.37. □

Exercise 1.7.55 Dongarra, Gustavson, and Karp [19] introduced the interesting concept of a *generic* Gaussian elimination algorithm. In this exercise we will develop the idea in the context of positive definite matrices. Let A be positive definite. Then by Theorem 1.7.33, A can be written as $A = MD^{-1}M^T$, where M is lower triangular, D is diagonal and positive definite, and M and D have the same main-diagonal entries.

(a) Rewrite the equation $A = MD^{-1}M^T$ in block form, where

$$A = \begin{bmatrix} a_{11} & b^T \\ b & \hat{A} \end{bmatrix}, \quad M = \begin{bmatrix} m_{11} & 0 \\ p & \tilde{M} \end{bmatrix}, \quad \text{and} \quad D = \begin{bmatrix} m_{11} & 0 \\ 0 & \tilde{D} \end{bmatrix}.$$

Show that $m_{11} = a_{11}$, $p = b$, and $\tilde{A} = \tilde{M}\tilde{D}^{-1}\tilde{M}^T$, if we define $\tilde{A} = \hat{A} - ba_{11}^{-1}b^T$. ($\tilde{A}$ is the Schur complement of a_{11} in A .)

(b) Use the result of part (a) to derive an outer-product formulation of the $MD^{-1}M^T$ decomposition. Write a pseudocode algorithm that accesses only the *lower* half of A and stores M over A .

Your solution should look something like this:

$$\begin{array}{l} \text{for } k = 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } j = k + 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } i = j, \dots, n \\ \quad \left[a_{ij} \leftarrow a_{ij} - \frac{a_{ik}a_{jk}}{a_{kk}} \end{array} \right. \end{array} \right. \end{array} \right. \end{array} \quad (1.7.56)$$

In practice there should be a check for positivity of a_{kk} at an appropriate point, but we have left it out to avoid clutter. The version shown here is the one obtained if the Schur complement calculation $\tilde{A} = \hat{A} - ba_{11}^{-1}b^T$ is done by

columns. If it is done by rows, the i and j loops should be interchanged:

$$\left[\begin{array}{l} \text{for } i = k + 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } j = k + 1, \dots, i \\ \quad \left[a_{ij} \leftarrow a_{ij} - \frac{a_{ik}a_{jk}}{a_{kk}} \end{array} \right. \end{array} \right. \quad (1.7.57)$$

- (c) Count the flops in (1.7.56), and note that there are $n^3/6$ divisions. Show how to cut the number of divisions down to about $n^2/2$ by introducing one extra temporary storage array of length n and moving the divisions out of the innermost loop. Once this has been done, the total flop count is about $n^3/3$, as one would expect.
- (d) Develop an inner-product formulation of the $MD^{-1}M^T$ algorithm.

Your solution should look something like this:

$$\left[\begin{array}{l} \text{for } j = 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } i = j, \dots, n \\ \quad \left[\begin{array}{l} \text{for } k = 1, \dots, j - 1 \\ \quad \left[a_{ij} \leftarrow a_{ij} - \frac{a_{ik}a_{jk}}{a_{kk}} \end{array} \right. \end{array} \right. \end{array} \right. \quad (1.7.58)$$

Again we have left out the positivity check for simplicity. Again one could move the divisions out of the inner loop. We have left them in the inner loop to keep the algorithm as uncluttered as possible. Notice that (1.7.56) and (1.7.58) are identical, except that the loops are nested differently. It is natural to ask whether one can obtain a valid $MD^{-1}M^T$ decomposition algorithm with the loop indices in any desired order. The answer turns out to be yes; one need only choose the ranges of the loop indices carefully. Thus we can write down a generic $MD^{-1}M^T$ algorithm

$$\left[\begin{array}{l} \text{for } \underline{\hspace{2cm}} \\ \quad \left[\begin{array}{l} \text{for } \underline{\hspace{2cm}} \\ \quad \left[\begin{array}{l} \text{for } \underline{\hspace{2cm}} \\ \quad \left[a_{ij} \leftarrow a_{ij} - \frac{a_{ik}a_{jk}}{a_{kk}} \end{array} \right. \end{array} \right. \end{array} \right. \quad (1.7.59)$$

The blanks can be filled with i , j , and k in any order. We can refer to the outer-product formulation (1.7.56) as the kji -algorithm, because kji is the order of the loops. If the j and i loops of (1.7.56) are replaced by (1.7.57), we get the kij -algorithm. The inner-product formulation (1.7.58) is the jik -algorithm. Since i , j , and k can be placed in any order, there are six basic algorithms. This is true not only for the $MD^{-1}M^T$ decomposition algorithms, but also for the LDL^T and Cholesky decomposition algorithms, since these can be obtained from the $MD^{-1}M^T$ algorithms by a few simple modifications.

- (e) Develop the bordered form of the $MD^{-1}M^T$ decomposition. Show that this is either the ijk -algorithm or the ikj -algorithm, depending whether the forward substitutions are done in a row-oriented or column-oriented manner.

- (f) We may fill in the loop indices i , j , and k in (1.7.59) in any order, provided that certain relationships between the variables i , j , k , and n are maintained. Determine these relationships, assuming (as above) that only the lower half of the array is used.

Solution: $1 \leq k < j \leq i \leq n$.

- (g) Now we have derived all of the variants except jki . Write down the jki -algorithm, starting from (1.7.59), being careful to get the ranges of the loop indices right.
- (h) Modify your algorithm from part (g) so that it computes the Cholesky decomposition instead of the $MD^{-1}M^T$ decomposition. Instead of computing M , compute G such that $A = GG^T$ (i.e. $G = R^T$). (*Outline:* You will need to take the square root of a_{jj} at the appropriate point to get g_{jj} . [Put in a positivity check before the square-root operation.] Then you will also need to put in a loop that divides a column by g_{jj} . Notice that this eliminates the need for the divisions in the inner-most loop.)

□

1.8 GAUSSIAN ELIMINATION WITH PIVOTING

We now begin the second phase of our study of the solution of $Ax = b$ by Gaussian elimination, in which we drop the assumption that the leading principal submatrices are nonsingular. We will develop an algorithm that uses elementary row operations of types 1 and 2 either to solve the system $Ax = b$ or (in theory, at least) to determine that it is singular. The algorithm is identical to the algorithm that we developed in Section 1.7, except that at each step a row interchange can be made. Thus we are now considering *Gaussian elimination with row interchanges*, which is also known as *Gaussian elimination with pivoting*. Let us consider the k th step of the algorithm. After $k - 1$ steps, the array that originally contained A has been transformed to the form

$$\left[\begin{array}{cccc|cccc} u_{11} & u_{12} & \cdots & u_{1,k-1} & u_{1k} & & \cdots & u_{1n} \\ m_{21} & u_{22} & \cdots & u_{2,k-1} & u_{2k} & & \cdots & u_{2n} \\ \vdots & & \ddots & \vdots & \vdots & & & \vdots \\ m_{k-1,1} & & & u_{k-1,k-1} & u_{k-1,k} & & \cdots & u_{k-1,n} \\ \hline m_{k1} & \cdots & & m_{k,k-1} & a_{k,k}^{(k-1)} & a_{k,k+1}^{(k-1)} & \cdots & a_{k,n}^{(k-1)} \\ & & & & a_{k+1,k}^{(k-1)} & a_{k+1,k+1}^{(k-1)} & \cdots & a_{k+1,n}^{(k-1)} \\ \vdots & & & \vdots & \vdots & \vdots & & \vdots \\ m_{n1} & \cdots & & m_{n,k-1} & a_{n,k}^{(k-1)} & a_{n,k+1}^{(k-1)} & \cdots & a_{n,n}^{(k-1)} \end{array} \right].$$

The u_{ij} are entries that will not undergo any further changes (eventual entries of the matrix U). The m_{ij} are stored multipliers (eventual entries of the matrix L). The $a_{ij}^{(k-1)}$ are the entries that are still active.

To calculate the multipliers for the k th step, we should divide by $a_{kk}^{(k-1)}$. If $a_{kk}^{(k-1)} = 0$, we will have to use a type 2 row operation (row interchange) to get a nonzero entry into the (k, k) position. In fact, even if $a_{kk}^{(k-1)} \neq 0$, we may still choose to do a row interchange.

Consider the following possibility. If $|a_{kk}^{(k-1)}|$ is very small, it may be that $a_{kk}^{(k-1)}$ should be exactly zero and is nonzero only because of roundoff errors made on previous steps. If we now calculate multipliers by dividing by this number, we will surely get erroneous results. For this and other reasons, we will always carry out row interchanges in such a way as to avoid having a small entry in the (k, k) position. The effects of roundoff errors will be studied in detail in Chapter 2.

Returning to the description of our algorithm, we examine the entries $a_{kk}^{(k-1)}$, $a_{k+1,k}^{(k-1)}, \dots, a_{nk}^{(k-1)}$. If all are zero, then A is singular (Exercise 1.8.1). Set a flag to warn that this is the case. At this point we can either stop or go on to step $k + 1$. If not all of $a_{kk}^{(k-1)}, \dots, a_{nk}^{(k-1)}$ are zero, let $a_{mk}^{(k-1)}$ be the one whose absolute value is greatest. Interchange rows m and k , including the stored multipliers. Keep a record of the row interchange. This is easily done in an integer array of length n . Store the number m in position k of the array to indicate that at step k , rows k and m were interchanged. Now subtract the appropriate multiples of the new k th row from rows $k + 1, \dots, n$ to produce zeros in positions $(k + 1, k), \dots, (n, k)$. Of course, we actually store the multipliers $m_{k+1,k}, \dots, m_{nk}$ in those positions instead of zeros. This concludes the description of step k .

Exercise 1.8.1 After $k - 1$ steps of the Gaussian elimination process, the coefficient matrix has been transformed to the form

$$B = \begin{bmatrix} B_{11} & B_{12} \\ 0 & B_{22} \end{bmatrix},$$

where B_{11} is $(k - 1) \times (k - 1)$ and upper triangular. Prove that B is singular if the first column of B_{22} is zero. $\square$

The eventual (k, k) entry, by which we divide to form the multipliers, is called the *pivot* for step k . The k th row, multiples of which are subtracted from each of the remaining rows at step k , is called the *pivotal row* for step k . Our strategy, which is to make the pivot at step k (i.e., at *each* step) as far from zero as possible in order to protect against disasters due to roundoff errors, is called *partial pivoting*. (Later on we will discuss *complete pivoting*, in which both rows and columns are interchanged.) Notice that the pivots end up on the main diagonal of the matrix U of the LU decomposition. Also, the choice of pivots implies that all of the multipliers will satisfy $|m_{ij}| \leq 1$. Thus in the matrix L of the LU decomposition, all entries will have absolute values less than or equal to 1.

After $n - 1$ steps, the decomposition is complete. One final check must be made: If $a_{nn}^{(n-1)} = 0$, A is singular; set a flag. This is the last pivot. It is not used to create zeros in other rows, but, being the (n, n) entry of U , it is used as a divisor in the back-substitution process.

Each time we did a row interchange, we interchanged the stored multipliers associated with those rows as well. Because of this, the effect of the row interchanges is the same as if the rows of A had been interchanged initially. In other words, suppose we took a copy of the original matrix A and, without doing any elimination operations, performed the same sequence of row interchanges as we did during the Gaussian elimination process. This would yield a matrix $\hat{A}$ that is just A with the rows scrambled. If we now carried out Gaussian elimination without row interchanges on $\hat{A}$, we would get exactly the same result as we got when we made the interchanges during the elimination process. Gaussian elimination without row interchanges yields an LU decomposition. Thus the result of our new algorithm is an LU decomposition of $\hat{A}$, not of A .

Solving the system $Ax = b$ is the same as solving a system $\hat{A}x = \hat{b}$, obtained by interchanging the equations. Since we have saved a record of the row interchanges, it is easy to permute the entries of b to obtain $\hat{b}$. We then solve $Ly = \hat{b}$ by forward substitution and $Ux = y$ by back substitution to get the solution vector x .

Example 1.8.2 We will solve the system

$$\begin{bmatrix} 0 & 4 & 1 \\ 1 & 1 & 3 \\ 2 & -2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 9 \\ 6 \\ -1 \end{bmatrix}$$

by Gaussian elimination with partial pivoting.

In step 1 the pivotal position is $(1, 1)$. Since there is a zero there, a row interchange is absolutely necessary. Since the largest potential pivot is the 2 in the $(3, 1)$ position, we interchange rows 1 and 3 to get

$$\begin{bmatrix} 2 & -2 & 1 \\ 1 & 1 & 3 \\ 0 & 4 & 1 \end{bmatrix}.$$

The multipliers for the first step are $1/2$ and 0. Subtracting $1/2$ the first row from the second row and storing the multipliers, we have

$$\begin{bmatrix} 2 & -2 & 1 \\ \frac{1}{2} & 2 & 5/2 \\ 0 & 4 & 1 \end{bmatrix}.$$

The pivotal position for the second step is $(2, 2)$. Since the 4 in the $(3, 2)$ position is larger than the 2 in the $(2, 2)$ position, we interchange rows 2 and 3 (including the multipliers) to get the 4 into the pivotal position:

$$\begin{bmatrix} 2 & -2 & 1 \\ 0 & 4 & 1 \\ \frac{1}{2} & 2 & 5/2 \end{bmatrix}.$$

The multiplier for the third step is $1/2$. Subtracting $1/2$ the second row from the third row and storing the multiplier, we have

$$\left[\begin{array}{ccc|c} 2 & -2 & 1 & \\ \hline 0 & 4 & 1 & \\ 1/2 & 1/2 & 2 & \end{array} \right]. \quad (1.8.3)$$

This completes the Gaussian elimination process. Noting that the pivot in the $(3, 3)$ position is nonzero, we conclude that A is nonsingular, and the system has a unique solution. Encoded in (1.8.3) is the LU decomposition of

$$\hat{A} = \begin{bmatrix} 2 & -2 & 1 \\ 0 & 4 & 1 \\ 1 & 1 & 3 \end{bmatrix},$$

which was obtained from A by making appropriate row interchanges. You can check that $\hat{A} = LU$, where

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 1/2 & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & -2 & 1 \\ 0 & 4 & 1 \\ 0 & 0 & 2 \end{bmatrix}.$$

To solve $Ax = b$, we first transform b to $\hat{b}$. Since we interchanged rows 1 and 3 at step 1, we must first interchange components 1 and 3 of b to obtain $[-1, 6, 9]^T$. At step 2 we interchanged rows 2 and 3, so we must interchange components 2 and 3 to get $\hat{b} = [-1, 9, 6]^T$. We now solve

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 1/2 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} -1 \\ 9 \\ 6 \end{bmatrix}$$

to get $y = [-1, 9, 2]^T$. Finally we solve

$$\begin{bmatrix} 2 & -2 & 1 \\ 0 & 4 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -1 \\ 9 \\ 2 \end{bmatrix}$$

to get $x = [1, 2, 1]^T$. You can easily check that this is correct by substituting it back into the original equation. $\square$

Exercise 1.8.4 Let

$$A = \begin{bmatrix} 2 & 2 & -4 \\ 1 & 1 & 5 \\ 1 & 3 & 6 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 10 \\ -2 \\ -5 \end{bmatrix}.$$

Use Gaussian elimination with partial pivoting (by hand) to find matrices L and U such that U is upper triangular, L is unit lower triangular with $|l_{ij}| \leq 1$ for all $i > j$, and $LU = \hat{A}$, where $\hat{A}$ can be obtained from A by making row interchanges. Use your LU decomposition to solve the system $Ax = b$. $\square$

Gaussian elimination with partial pivoting works very well in practice, but it is important to realize that an accurate answer is not absolutely guaranteed. There exist ill-conditioned systems (Chapter 2) that simply cannot be solved accurately in the presence of roundoff errors. The most extreme case of an ill-conditioned system is one whose coefficient matrix is singular; there is no unique solution to such a system. Our algorithm is supposed to detect this case, but unfortunately it usually will not. If the coefficient matrix is singular, there will be a step at which all potential pivots $a_{kk}^{(k-1)}, \dots, a_{nk}^{(k-1)}$ are zero. However, due to roundoff errors on previous steps, the actual computed quantities will not be exact zeros. Consequently the singularity of the matrix will not be detected, and the algorithm will march ahead and produce a nonsensical “solution.” An obvious precautionary measure would be to issue a warning whenever the algorithm is forced to use a very small pivot. Better ways to detect inaccuracy will be discussed in Chapter 2.

The additional costs associated with row interchanges in the partial-pivoting strategy are not great. There are two costs to consider, that of finding the largest pivot at each step, and that of physically interchanging the rows. At the k th step, $n - k + 1$ numbers must be compared to find the one that is largest in magnitude. This does not involve any arithmetic, but it requires the comparison of $n - k$ pairs of numbers. The total number of comparisons made in the $n - 1$ steps is therefore

$$\sum_{k=1}^{n-1} (n - k) = \frac{n(n-1)}{2} \approx \frac{n^2}{2}.$$

Regardless of the exact cost of making one comparison, the cost of making $n^2/2$ comparisons is small compared with the $O(n^3)$ arithmetic cost of the algorithm. The interchange of rows does not require any arithmetic either, but it takes time to fetch and store numbers. At each step, at most one row interchange is carried out. This involves fetching and storing $2n$ numbers. Since there are $n - 1$ steps, the total number of fetches and stores is not more than about $2n^2$. Thus the cost of interchanging rows is also negligible compared with the cost of the arithmetic of the algorithm. It is fair to say, as a rough approximation, that the cost of Gaussian elimination with partial pivoting is $\frac{2}{3}n^3$ flops.

Permuting the rows of a matrix is equivalent to multiplying the matrix by a permutation matrix.

Example 1.8.5 Let A and $\hat{A}$ be as in Example 1.8.2. Then one easily checks that $\hat{A} = PA$, where

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

□

A *permutation matrix* is a matrix that has exactly one 1 in each row and in each column, all other entries being zero. For example, the matrix P in Example 1.8.5 is a permutation matrix.

Exercise 1.8.6 Let A be an $n \times m$ matrix, and let $\hat{A}$ be a matrix obtained from A by scrambling the rows. Show that there is a unique $n \times n$ permutation matrix P such that $\hat{A} = PA$. $\square$

Exercise 1.8.7 Show that if P is a permutation matrix, then $P^T P = P P^T = I$. Thus P is nonsingular, and $P^{-1} = P^T$. $\square$

We now easily obtain the following result.

Theorem 1.8.8 *Gaussian elimination with partial pivoting, applied to an $n \times n$ matrix A produces a unit lower-triangular matrix L such that $|l_{ij}| \leq 1$, an upper triangular matrix U , and a permutation matrix P such that*

$$A = P^T L U.$$

Proof. We have seen that Gaussian elimination with partial pivoting produces L and U of the required form such that $\hat{A} = LU$, where $\hat{A}$ was obtained from A by permuting the rows. By Exercise 1.8.6, there is a permutation matrix P such that $\hat{A} = PA$. Thus $PA = LU$ or $A = P^{-1}LU = P^T LU$. $\square$

Exercise 1.8.9 Let A be the matrix in Exercise 1.8.4. Determine matrices P , L , and U with the properties stated in Theorem 1.8.8, such that $A = P^T LU$. $\square$

Given a decomposition $A = P^T LU$, we can use it to solve a system $Ax = b$, by writing $P^T LUx = b$, then solving, successively, $P^T \hat{b} = b$, $Ly = \hat{b}$, and $Ux = y$. The step $P^T \hat{b} = b$, or $\hat{b} = Pb$, just amounts to rearranging the entries of b in accordance with the row interchanges that were carried out during the elimination process. Thus this process is really no different from the process illustrated at the end of Example 1.8.2. In practice there is never really any need to build the permutation matrix P . Keeping a record of the row interchanges is just as good and much more compact. However, MATLAB will display P for you if you ask it to.

Earlier in the chapter there were quite a few exercises that used MATLAB to solve systems of linear equations. Whenever the command `x=A\b` is invoked to solve the system $Ax = b$ (where A is $n \times n$ and nonsingular), MATLAB uses Gaussian elimination with partial pivoting to solve the system. If you want to see the LU decomposition, use the MATLAB command `lu`.

Exercise 1.8.10 Use MATLAB to check the LU decomposition obtained in Example 1.8.2. Enter the matrix A , then type `[L,U,P] = lu(A)`. Once you have L , U , and P , you can put them back together by typing `P'*L*U`. (In MATLAB the prime symbol, applied to a real matrix, means “transpose.”) You can also try `[K,V] = lu(A)`, and see what that gives you. Give a matrix equation that relates K to L . For a description of the `lu` command, type `help lu`. $\square$

Exercise 1.8.11 Repeat Exercise 1.8.10 using the matrix from Exercises 1.8.4 and 1.8.9. $\square$

Programming Gaussian Elimination

Exercise 1.8.12 Write an algorithm that implements Gaussian elimination with partial pivoting. Store L and U over A , and save a record of the row interchanges. $\square$

Your solution to Exercise 1.8.12 should look something like this:

```

Gauss ( $A, n, \text{intch}, \text{flag}$ )
clear  $\text{flag}$ 
for  $k = 1, 2, \dots, n - 1$ 
     $\left[ \begin{array}{l} \text{amax} \leftarrow \max\{|a_{kk}|, |a_{k+1,k}|, \dots, |a_{nk}|\} \\ \text{if } (\text{amax} = 0) \text{ then} \\ \quad \left[ \begin{array}{l} \text{set } \text{flag} \\ \text{intch}(k) \leftarrow 0 \end{array} \right] \quad (A \text{ is singular}) \\ \text{else} \\ \quad \left[ \begin{array}{l} m \leftarrow \text{smallest integer } \geq k \text{ for which } |a_{mk}| = \text{amax} \\ \text{intch}(k) \leftarrow m \quad (\text{record row interchange}) \\ \text{if } (m \neq k) \text{ then} \\ \quad \left[ \text{interchange rows } k \text{ and } m, \text{ including multipliers} \right] \\ \text{for } i = k + 1, \dots, n \\ \quad \left[ \begin{array}{l} a_{ik} \leftarrow a_{ik}/a_{kk} \quad \text{calculate multipliers} \\ \text{for } i = k + 1, \dots, n \\ \quad \left[ \begin{array}{l} \text{subtract } a_{ik} \text{ times } [a_{k,k+1}, \dots, a_{k,n}] \\ \text{from } [a_{i,k+1}, \dots, a_{i,n}] \end{array} \right] \end{array} \right] \end{array} \right] \quad (\text{row operations}) \end{array} \right.$ 
    if ( $a_{nn} = 0$ ) then
         $\left[ \begin{array}{l} \text{set } \text{flag} \\ \text{intch}(n) \leftarrow 0 \end{array} \right] \quad (A \text{ is singular})$ 
    else
         $\left[ \text{intch}(n) \leftarrow n \right]$ 

```

If A is found to be singular, the algorithm sets a flag but does not stop. It finishes the LU decomposition, but U is singular. The array A now contains the LU decomposition of $\hat{A} = PA$, and the array intch contains a record of the row interchanges (i.e. an encoding of the permutation matrix P).

The bulk of the work is in the row operations. These can be organized several ways. If each row operation is executed as a loop, we have

$$\begin{array}{l} \text{for } i = k + 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } j = k + 1, \dots, n \\ \quad [a_{ij} \leftarrow a_{ij} - a_{ik}a_{kj} \end{array} \right] \end{array} \quad (1.8.13)$$

Each time through the outer loop, a complete row operation is performed. However there is no logical reason why the j loop should be on the inside. Clearly the code segment

$$\begin{array}{l} \text{for } j = k + 1, \dots, n \\ \quad \left[\begin{array}{l} \text{for } i = k + 1, \dots, n \\ \quad [a_{ij} \leftarrow a_{ij} - a_{ik}a_{kj} \end{array} \right] \end{array} \quad (1.8.14)$$

performs exactly the same operations, and there is no a priori reason to prefer (1.8.13) over (1.8.14). If (1.8.13) is used, the algorithm is said to be *row oriented*, because in the inner loop the row index i is held fixed. If (1.8.14) is used, the algorithm is said to be *column oriented*. In this case the elements a_{ij} and a_{ik} traverse the j th and k th columns, respectively, as the row index i is stepped forward.

Which orientation is best depends on the language in which the algorithm is implemented. If Fortran is used, for example, the column-oriented code is generally faster, because Fortran stores arrays by columns. As each column is traversed, elements are taken from consecutive locations in the computer memory.

In order to solve systems of linear equations, we need not only an LU decomposition routine, but also a routine to permute the b vector and perform forward and back substitution. Such a routine would look something like this:

```
Solve ( $A, b, n, intch, flag$ )
for  $k = 1, \dots, n - 1$ 
    [  $m \leftarrow intch(k)$ 
      interchange  $b_k$  and  $b_m$ 
    ]
for  $j = 1, \dots, n - 1$ 
    [ for  $i = j + 1, \dots, n$ 
      [  $b_i \leftarrow b_i - a_{ij}b_j$  ] } (column-oriented forward substitution)
clear  $flag$ 
for  $j = n, n - 1, \dots, 1$ 
    [ if ( $a_{jj} = 0$ ) then
      [ set  $flag$ 
        exit ] } ( $U$  is singular)
       $b_j \leftarrow b_j / a_{jj}$ 
      for  $i = 1, \dots, j - 1$ 
        [  $b_i \leftarrow b_i - a_{ij}b_j$  ] } (column-oriented back substitution)
```

This algorithm takes as inputs the outputs A and $intch$ from the algorithm Gauss. The array A contains an LU decomposition, and $intch$ contains a record of the row interchanges. The array b contains the vector b initially and the solution vector x afterward.

A main program to drive the subroutines Gauss and Solve might be organized something like this:

```
read  $n$  and  $A$ 
call Gauss ( $A, n, intch, flag$ )
if (flag is set) then
    [ print 'coefficient matrix is singular'
      stop ]
read  $num$  (number of right-hand sides)
for  $m = 1, \dots, num$ 
    [ read  $b$ 
      call Solve ( $A, b, n, intch, flag$ )
      print 'solution number'  $m$  'is'  $b$  ] (prints solution)
```

This program allows for the possibility that the user would like to solve several problems $Ax = b^{(1)}$, $Ax = b^{(2)}$, ..., $Ax = b^{(k)}$ with the same coefficient matrix. The LU decomposition is only done once.

Exercise 1.8.15 Write a Fortran program and subroutines to solve linear systems $Ax = b$ by Gaussian elimination with partial pivoting. Your subroutines should be column oriented. Try out your program on the test problems $Ax = b$ and $Ax = c$, where

$$A = \begin{bmatrix} 2 & 10 & 8 & 8 & 6 \\ 1 & 4 & -2 & 4 & -1 \\ 0 & 2 & 3 & 2 & 1 \\ 3 & 8 & 3 & 10 & 9 \\ 1 & 4 & 1 & 2 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 52 \\ 14 \\ 12 \\ 51 \\ 15 \end{bmatrix} \quad c = \begin{bmatrix} 50 \\ 4 \\ 12 \\ 48 \\ 12 \end{bmatrix}.$$

After the decomposition your transformed matrix should be

$$\begin{bmatrix} 3 & 8 & 3 & 10 & 9 \\ \hline 2/3 & 14/3 & 6 & 4/3 & 0 \\ 1/3 & 2/7 & -33/7 & 2/7 & -4 \\ \hline 1/3 & 2/7 & 4/11 & -20/11 & -6/11 \\ 0 & 3/7 & -1/11 & -4/5 & 1/5 \end{bmatrix}.$$

The row interchanges are given by $intch = [4, 4, 4, 5, 5]$. The solution vectors are $[1, 2, 1, 2, 1]^T$ and $[2, 1, 2, 1, 2]^T$. Once you have your program working this problem correctly, try it out on some other problems of your own devising. $\square$

Exercise 1.8.16 Write a program in your favorite computer language to solve systems of linear equations by Gaussian elimination with partial pivoting. Determine whether your language stores arrays by rows or by columns, and write your subroutines accordingly. Test your program on the problem from Exercise 1.8.15 and other problems of your own devising. $\square$

Exercise 1.8.17 Write two versions of your Gaussian elimination program, one that is row oriented and one that is column oriented. Time them on some big problems (using randomly-generated coefficient matrices, for example.) Which is faster? Is this what you expected? $\square$

Some modern languages, for example Fortran-90, allow the programmer to leave the choice of row-oriented or column-oriented code to the compiler. The row operations (1.8.13) or (1.8.14) amount to an outer-product update (This is the outer-product formulation)

$$\hat{A} \leftarrow \hat{A} - vw^T, \quad (1.8.18)$$

where

$$\hat{A} = \begin{bmatrix} a_{k+1,k+1} & \cdots & a_{k+1,n} \\ \vdots & & \vdots \\ a_{n,k+1} & \cdots & a_{nn} \end{bmatrix}, \quad v = \begin{bmatrix} a_{k+1,k} \\ \vdots \\ a_{nk} \end{bmatrix}, \quad \text{and} \quad w = \begin{bmatrix} a_{k,k+1} \\ \vdots \\ a_{kn} \end{bmatrix}.$$

Fortran-90 allows matrix operations on arrays or sections of arrays. The submatrix $\hat{A}$ is denoted $a(k+1:n, k+1:n)$ in Fortran 90 (just as in MATLAB!), and the update (1.8.18) is given by

$$a(k+1:n, k+1:n) = a(k+1:n, k+1:n) - \text{matmul}(a(k+1:n, k:k), a(k:k, k+1:n)) \quad (1.8.19)$$

The `matmul` command performs matrix multiplication. This single instruction tells the computer to perform $2(n-k)^2$ flops. It is left to the Fortran-90 compiler to decide on the order in which the flops are performed. This is certainly more compact than either (1.8.13) or (1.8.14). It may be faster or slower, depending on how well the `matmul` command has been implemented in your compiler on your machine.

Our introduction to row interchanges has focused on the outer-product formulation of Gaussian elimination. However, row interchanges can easily be incorporated into other formulations of the *LU* decomposition algorithm (e.g. inner-product formulation, bordered form) as well. Thus there is a great variety of ways to implement Gaussian elimination with partial pivoting.

For larger matrices the fastest codes are those that make effective use of cache. To this end, we should perform Gaussian elimination by blocks. The advantages of blocking were discussed in the context of matrix multiplication in Section 1.1 and again in the context of the Cholesky decomposition in Section 1.4. A block Gaussian elimination algorithm can be built using the ideas discussed in Exercise 1.7.50 and in the discussion of block variants of Cholesky's method in Section 1.4. Every version of Gaussian elimination has a block variant. The need to perform row interchanges complicates the procedures but does not change the basic ideas.

Reliable Fortran routines that perform Gaussian elimination with partial pivoting can be obtained from the public-domain software packages LINPACK and LAPACK [1]. The LAPACK codes are organized by blocks. There is also a C version of LAPACK called CLAPACK.

As we have seen, there are many ways to organize Gaussian elimination with partial pivoting. There are many more details than have been mentioned here, as one sees by examining some of the codes from LINPACK or LAPACK.

Exercise 1.8.20 Visit the NETLIB repository on the Worldwide Web (<http://www.netlib.org>). Browse the repository and find the LINPACK and LAPACK libraries.

- (a) Find the LINPACK subroutine `dgefa`, which does Gaussian elimination with partial pivoting in a column-oriented manner. Make a copy of the subroutine and annotate it, telling what each part does. Notice that most of the work is done by the subroutines `idamax`, `dscal`, and (especially) `daxpy`. These are Basic Linear Algebra Subroutines or BLAS. Find the BLAS library to figure out what these subroutines do.
- (b) Find the LAPACK subroutine `dgetrf`. This does Gaussian elimination using scalar code (`dgetf2`) or by blocks, depending on the size of the matrix. If the elimination is done by blocks, the work is done by level-3 BLAS, subroutines that perform matrix operations. Which one does the bulk of the work?

□

Calculating A^{-1}

Your program to solve $Ax = b$ can be used to calculate the inverse of a matrix. Letting $X = A^{-1}$, we have $AX = I$. Rewriting this equation in partitioned form as

$$A[x_1 \ x_2 \ \cdots \ x_n] = [e_1 \ e_2 \ \cdots \ e_n],$$

where $x_1, \dots, x_n$ and $e_1, \dots, e_n$ are the columns of X and I , respectively, we find that the equation $AX = I$ is equivalent to the n equations

$$Ax_i = e_i, \quad i = 1, \dots, n. \quad (1.8.21)$$

Solving these n systems by Gaussian elimination with partial pivoting, we obtain A^{-1} .

Exercise 1.8.22 Use your program to find A^{-1} , where A is the matrix in Exercise 1.8.15. □

How much does it cost to calculate A^{-1} ? The LU decomposition has to be done once, at a cost of $\frac{2}{3}n^3$ flops. Each of the n systems in (1.8.21) has to be solved by forward and back substitution at a cost of $2n^2$ flops. Thus the total flop count is $\frac{8}{3}n^3$. A more careful count shows that the job can be done for a bit less.

Exercise 1.8.23 The forward-substitution phase requires the solution of $Ly_i = e_i$, $i = 1, \dots, n$. Some operations can be saved by exploiting the leading zeros in e_i (See Section 1.3). Do a flop count that takes these savings into account, and conclude that A^{-1} can be computed in about $2n^3$ flops. □

Exercise 1.8.24 Modify your Gaussian elimination program so that the forward substitution segment exploits leading zeros in the right-hand side. You now have a program that can calculate an inverse in $2n^3$ flops. □

You may be wondering how this method compares with other methods of calculating A^{-1} that you have seen. One method that is sometimes presented in elementary linear algebra classes starts with an augmented matrix $[A \mid I]$ and performs row operations that transform the augmented matrix to the form $[I \mid X]$. That algorithm also takes $2n^3$ flops, if it is organized efficiently. In fact it is basically the same algorithm as has been proposed here. Another method you may have seen is the cofactor method, which is summarized by the equation

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A).$$

That method requires the computation of many determinants. If the determinants are calculated in the classical way (row or column expansion), the cost is more than $n!$ flops. Since $n!$ grows much more rapidly than n^3 , the cofactor method is not competitive unless n is very small.

By the way, there are ways to compute the determinants in fewer than $n!$ flops. One very good way is to calculate an LU decomposition. Then $\det(A) = \pm \det(U) = \pm u_{11}u_{22} \cdots u_{nn}$, where the sign is minus (plus) if and only if an odd (even) number of row interchanges was made. This method obviously costs $\frac{2}{3}n^3$ flops.

Exercise 1.8.25 Verify that this procedure does indeed yield the determinant of A . □

MATLAB has commands `inv` and `det` that will compute the inverse and determinant for you.

Complete Pivoting

A more cautious pivoting strategy known as *complete pivoting* deserves at least brief mention. Both row and column interchanges are allowed. At the first step the entire matrix is searched. The element of largest magnitude is found and moved to the $(1,1)$ position by a row interchange and a column interchange. (Note that the effect of an interchange of columns i and j is to interchange the unknowns x_i and x_j .) The maximal element is then used as a pivot to create zeros below it. The second step is the same as the first, but it operates on the $(n-1) \times (n-1)$ submatrix obtained by ignoring the first row and column, and so on. The complete pivoting strategy gives extra protection against the bad effects of roundoff errors and is quite satisfactory both in theory and in practice. Its disadvantage is that it is somewhat more expensive than partial pivoting. In the first step about n^2 pairs of numbers have to be compared in order to find the largest entry. In the second step, $(n-1)^2$ comparisons are required, in the third step $(n-2)^2$, and so on. Thus the total number of comparisons made during the pivot searches is approximately $\sum_{k=0}^{n-2} (n-k)^2 \approx \frac{1}{3}n^3$. Since the cost of making a comparison is not insignificant, this means that the cost of the pivot searches is of the same order of magnitude as the cost of the arithmetic. In contrast the total cost of the pivot searches in the partial-pivoting strategy is $O(n^2)$ and is therefore insignificant if n is large. The extra cost of complete pivoting would be worth paying if it gave significantly better results. However, it has been found that partial pivoting works very satisfactorily in practice, so partial pivoting is much more widely used.

Exercise 1.8.26 Let A be an $n \times m$ matrix, and let $\hat{A}$ be a matrix obtained by shuffling the *columns* of A . Express the relationship between A and $\hat{A}$ as a matrix equation involving a permutation matrix. □

Exercise 1.8.27 (a) Show that Gaussian elimination with complete pivoting, applied to a nonsingular matrix A , produces unit lower-triangular L , upper-triangular U , and permutation matrices P and Q , such that

$$A = P^T L U Q^T.$$

(b) What additional properties do L and U satisfy? (c) In solving a system $Ax = b$ by Gaussian elimination with complete pivoting, an additional final step $x = Q\hat{x}$ must be carried out. How should this be done in practice? $\square$

On Competing Methods

A variant of Gaussian elimination that is taught in some elementary linear algebra textbooks is *Gauss-Jordan elimination*. In this variant the augmented matrix $[A \mid b]$ is converted to the form $[I \mid x]$ by elementary row operations. At the k th step the pivot is used to create zeros in column k both above and below the main diagonal. The disadvantage of this method is that it is more costly than the variants that we have discussed.

Exercise 1.8.28 (a) Do a flop count for Gauss-Jordan elimination and show that about n^3 flops are required, which is 50% more than ordinary Gaussian elimination needs. (b) Why is Gauss-Jordan elimination more expensive than transformation to the form $[U \mid y]$, followed by back substitution? (c) How could the operations in Gauss-Jordan elimination be reorganized so that only $\frac{2}{3}n^3$ flops are needed to get to the form $[I \mid x]$? $\square$

Another method for solving $Ax = b$ that was at one time covered in most elementary linear algebra texts is Cramer's rule, which is closely related to the cofactor method of calculating A^{-1} . Cramer's rule states that each entry x_i in the solution of $Ax = b$ is a quotient of two determinants:

$$x_i = \frac{\det(A^{(i)})}{\det(A)} \quad i = 1, \dots, n,$$

where $A^{(i)}$ denotes the matrix obtained from A by replacing its i th column by b . This truly elegant formula is too expensive to be a practical computational tool, except when $n = 2$ or 3 , because it requires the computation of determinants.

At the end of Section 1.1 we mentioned Strassen's method [68] and other methods that multiply two $n \times n$ matrices together in fewer than $O(n^3)$ flops. For example, Strassen's method takes $O(n^s)$ flops, where $s = \log_2 7 \approx 2.81$. These methods can be modified so that they compute A^{-1} . Therefore there exist methods that solve $Ax = b$ in fewer than $O(n^3)$ flops. Some of these methods could become important for large, dense (i.e. not sparse) matrix computations at some future date.

All of the methods that we have discussed so far are *direct methods*; if they were executed in exact arithmetic, they would produce the exact solution to $Ax = b$ after a finite, prespecified sequence of operations. A completely different type of method is the *iterative method*, which produces a sequence of successively better approximations to the solution. For extremely large, sparse matrices, iterative methods are the best choice. We discuss iterative methods in Chapter 7.